

PROGETTO BASI DI DATI

Componenti del gruppo:

1) Matteo Boscherini	0001079114
2) Alessandro Campedelli	0001077616
3) Nicolas Cola	0001080556

INDICE	PAG.
MODELLO ER	2
TABELLA DELLE ENTITÀ	2
TABELLA DELLE RELAZIONI	4
TABELLA DELLE BUSINESS RULES	5
ANALISI DELLA RIDONDANZA	7
CONCLUSIONE ANALISI DELLA RIDONDANZA	9
NORMALIZZAZIONE	9
MODELLO ER RISTRUTTURATO	10
MODELLO LOGICO	10
MANUALE	12
CODICE SQL	42
ISTRUZIONI D'USO	55

MODELLO ER

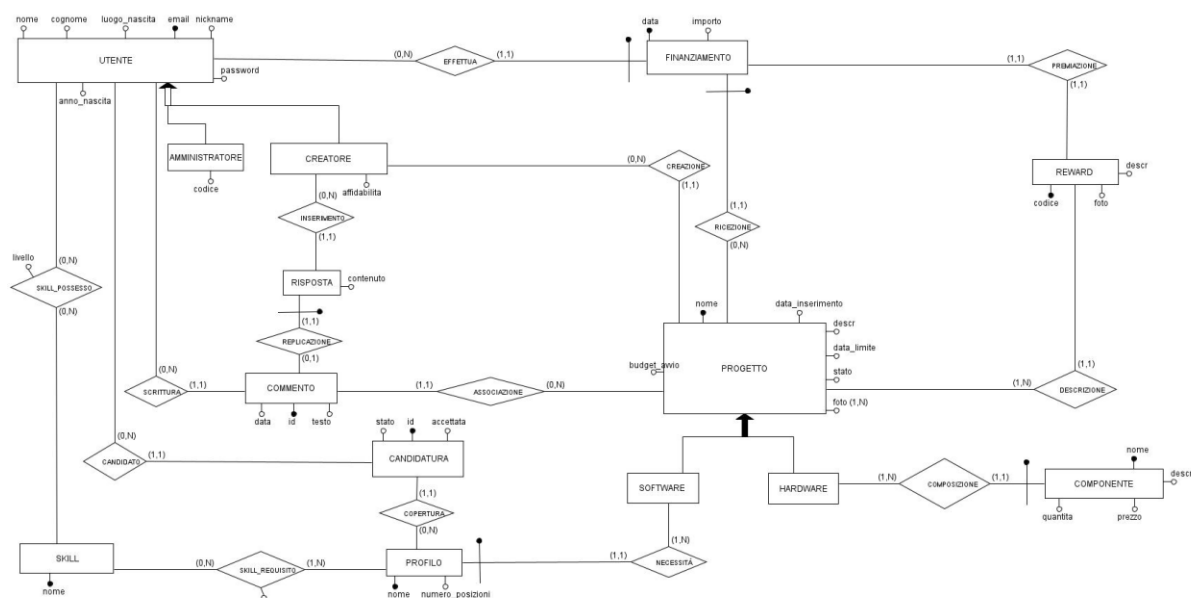


TABELLA DELLE ENTITA'

ENTITA'	ATTRIBUTI	DESCRIZIONE
Utente	email nome cognome luogo_nascita anno_nascita nickname password	Rappresenta un utente registrato nel sistema.
Amministratore	codice	Identifica un amministratore del sistema.
Creatore	affidabilita	Identifica un utente che crea progetti.
Finanziamento	data emailUtente nomeProgetto importo	Traccia un finanziamento fatto da un utente ad un progetto.

Reward	codice foto descr	Descrive una ricompensa associata ad un progetto.
Skill	nome	Rappresenta una specifica competenza.
Profilo	nome nomeProgetto numero_posizioni	Definisce un profilo richiesto in un progetto.
Candidatura	id stato accettata	Traccia una candidatura per un ruolo in un progetto.
Commento	id data testo	Rappresenta un commento fatto su un progetto.
Risposta	idCommento contenuto	Rappresenta la risposta ad un commento.
Progetto	nome data_inserimento data_limite descr stato budget_avvio foto (1-N)	Descrive un progetto creato nel sistema.
Hardware		Specifica un progetto di tipo hardware.
Software		Specifica un progetto di tipo software.
Componente	nome nomeProgetto prezzo descr quantita	Dettaglia i componenti necessari ad un progetto di tipo hardware.

TABELLA DELLE RELAZIONI

RELAZIONE	ENTITA'	ATTRIBUTI	DESCRIZIONE
Effettua (1-N)	Utente <-> Finanziamento	/	Gestisce il collegamento tra un utente e un finanziamento fornito.
Premiazione (1-1)	Finanziamento <-> Reward	/	Gestisce l'assegnazione di un reward a un finanziamento.
Ricezione (1-N)	Finanziamento <-> Progetto	/	Rappresenta il finanziamento ricevuto per un progetto.
Descrizione (1-N)	Reward <-> Progetto	/	Rappresenta la descrizione di un progetto associato ad una reward.
Associazione (1-N)	Commento <-> Progetto	/	Collega un commento ad uno specifico progetto.
Scrittura (1-N)	Utente <-> Commento	/	Traccia l'azione di un utente che scrive un commento.
Inserimento (1-N)	Creatore <-> Risposta	/	Rappresenta l'invio di una risposta da parte di un creatore.
Replicazione (1-1)	Risposta <-> Commento	/	Gestisce la relazione tra una risposta e il commento a cui essa è collegata.
Creazione (1-N)	Creatore <->	/	Rappresenta la

	Progetto		creazione di un progetto da parte di un creatore.
Composizione (1-N)	Hardware <-> Componente	/	Specifica la relazione tra un componente hardware e la sua composizione.
Necessità (1-N)	Software <-> Profilo	/	Traccia la richiesta di un software da parte di un profilo utente.
Copertura (1-N)	Candidatura <-> Profilo	/	Collega un profilo ad una candidatura ricevuta.
Skill_Possesso (N-N)	Utente <-> Skill	livello	Traccia il possesso di una skill da parte di un utente.
Skill_Requisito (N-N)	Skill <-> Profilo	livello	Gestisce il collegamento tra un profilo e una skill richiesta includendo il livello richiesto.

TABELLA DELLE BUSINESS RULES

VINCOLO	DESCRIZIONE
nome (Skill)	La lista delle competenze è comune a tutti gli utenti della piattaforma.
Amministratore, Skill	Solo gli utenti amministratori possono popolare la lista delle competenze.
quantita (Componente)	L'attributo quantita dell'entità Componente deve essere maggiore di 0.
numero_posizioni	L'attributo numero_posizioni dell'entità Profilo

(Profilo)	deve essere maggiore di 0.
livello (Skill_Possesso)	L'attributo livello nella relazione Skill_Possesso deve essere compreso tra 0 e 5.
livello (Skill_Requisito)	L'attributo livello nella relazione Skill_Requisito deve essere compreso tra 0 e 5.
stato (Progetto)	L'attributo stato dell'entità Progetto è un enum con valore "aperto" o "chiuso".
stato (Progetto)	Un progetto in stato "chiuso" non accetta ulteriori finanziamenti.
Finanziamento, Progetto	Nel momento in cui la somma totale degli importi dei finanziamenti per un progetto supera il budget di quest'ultimo, lo stato del progetto diventa "chiuso".
Candidatura, Profilo	Un utente può inserire una candidatura per un profilo solo se, per ogni skill richiesta da quest'ultimo, l'utente dispone di un livello superiore o uguale a quello richiesto.
Creatore, Candidatura	L'utente creatore di un progetto software può decidere se accettare o meno le candidature ricevute per un profilo.
Candidatura, Progetto	Un utente può inviare più candidature per uno stesso profilo di uno stesso progetto se lo stato delle candidature precedenti è "rifiutata".
affidabilità (Creatore)	Una volta registrato come creatore, l'affidabilità è impostata a 0.

ANALISI DELLA RIDONDANZA

Coefficienti per l'analisi

- Peso operazioni scrittura (α) = 2
- Peso operazioni interattive (w_I) = 1
- Peso operazioni batch (w_B) = 0.5

Tabella dei volumi

- 10 progetti con 3 finanziamenti per progetto
- 5 utenti con 2 progetti per utente

Calcoli Costi (senza ridondanza "num_progetti" nell'entità "Creatore")

OP1: Aggiungere un nuovo progetto ad un utente creatore esistente

Tipo: Interattiva con Frequenza: 1 volta/mese

Scrittura: 2 operazioni

$$C(OP1) = f(OP1) * w_I * (\alpha * N(\text{scrittura})) = 1 * 1 * (2 * 2) = 1 * (4) = 4$$

OP2: Visualizzare tutti i progetti e tutti i finanziamenti

Tipo: Batch con Frequenza: 1 volta/mese

Lettura: 10 progetti e 30 finanziamenti.

$$C(OP2) = f(OP2) * w_B * (N(\text{lettura})) = 1 * 0.5 * (10 + (10*3)) = 0.5 * (10 + 30) \\ = 0.5 * (40) = 20$$

OP3: Contare il numero di progetti associati a uno specifico utente

Tipo: Batch con Frequenza: 3 volta/mese

Lettura: 1 utente con 2 progetti = 2 operazioni

$$C(OP3) = f(OP3) * w_B * (N(\text{lettura})) = 3 * 0.5 * (2) = 1.5 * (2) = 3$$

Somma dei costi senza ridondanza: C senza ridondanza = $4 + 20 + 3 = 27$

Calcoli Costi (con ridondanza "num_progetti" nell'entità "Creatore")

La presenza della ridondanza “num_progetti” nell'entità "Creatore" **non influisce** sull'operazione **2**, ma ha un **impatto marginale** sull'operazione **1** e sull'operazione **3**.

OP1: Aggiungere un nuovo progetto ad un utente creatore esistente

Tipo: Interattiva con Frequenza: 1 volta/mese

Scrittura: 3 operazioni

(con la ridondanza, ogni operazione di aggiunta comporta un'ulteriore operazione di scrittura per aggiornare il contatore di 1 dell'attributo “num_progetti” del creatore)

$$C(OP1) = f(OP1) * wI * (alfa * N(scrittura)) = 1 * 1 * (2 * 3) = 1 * (6) = 6$$

OP2: Visualizzare tutti i progetti e tutti i finanziamenti

Tipo: Batch con Frequenza: 1 volta/mese

Lettura: 10 progetti e 30 finanziamenti.

$$C(OP2) = f(OP2) * wB * (N(lettura)) = 1 * 0.5 * (10 + (10*3)) = 0.5 * (10 + 30) = 0.5 * (40) = 20$$

OP3: Contare il numero di progetti associati a uno specifico utente.

Tipo: Batch con Frequenza: 3 volta/mese

Lettura: 1 operazione

(con la ridondanza, il conteggio non richiede più di contare il numero di progetti di ogni utente, ma solo 1 lettura dell'attributo "num_progetti")

$$C(OP3) = f(OP3) * wB * (N(lettura)) = 3 * 0.5 * (1) = 1.5 * (1) = 1.5$$

Somma dei costi con ridondanza: C con ridondanza = 6 + 20 + 1.5 = 27.5

CONFRONTO FINALE

Confronto tra costi: C senza ridondanza / C con ridondanza = 27 / 27.5 = 0.98

Essendo $0.98 < 1$, conviene **rimuovere** la **ridondanza concettuale** "num_progetti".

IMPATTO SULLA MEMORIA

Essendo 5 utenti, il costo dell'occupazione in memoria per l'aggiunta di un attributo (in questo caso, la ridondanza "num_progetti") è di 4 byte per progetto. In totale, per 5 utenti, abbiamo $4 \times 5 = 20$ byte. Pertanto, la memoria occupata sarà: $X + 20$. Essendo una differenza poco rilevante e non significativa, si ritiene efficiente **rimuovere** la **ridondanza**.

CONCLUSIONE ANALISI DELLA RIDONDANZA

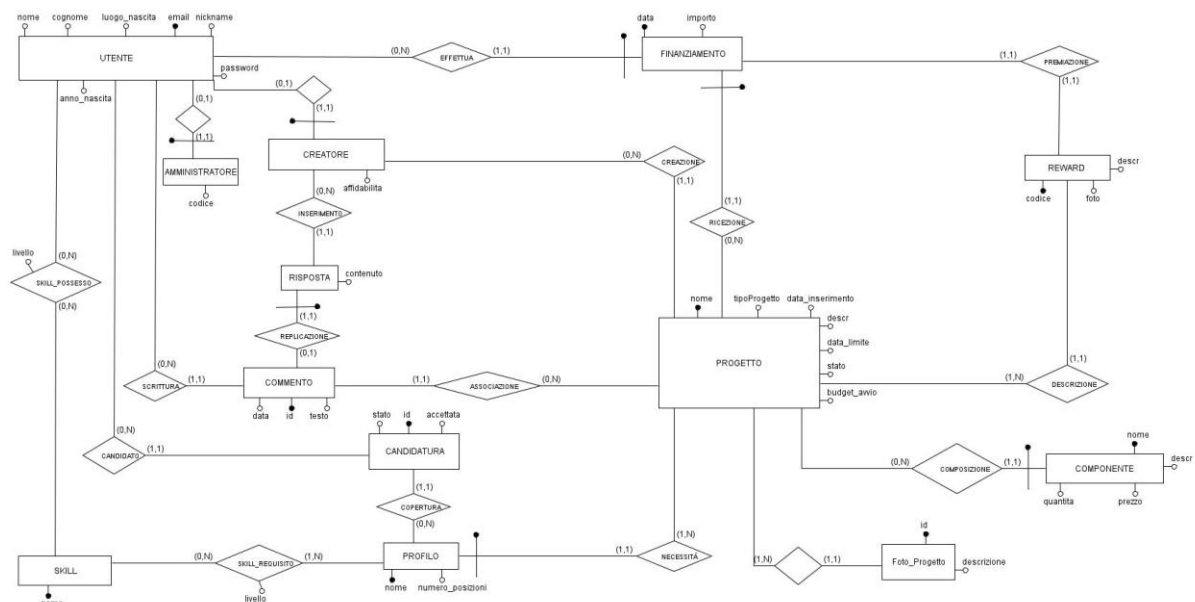
In conclusione, conviene **rimuovere** la ridondanza concettuale "nr_progetti". L'analisi dimostra che il costo relativo all'efficienza della ridondanza è minore di 1 (0.98), indicando che la ridondanza è **svantaggiosa**.

NORMALIZZAZIONE

Ogni tabella dello schema possiede una **chiave primaria minimale**, ovvero il più piccolo insieme di attributi in grado di identificare univocamente ogni tupla. Inoltre, tutte le dipendenze funzionali presenti dipendono esclusivamente dalla chiave primaria: se i valori della chiave si ripetono, anche quelli degli altri attributi rimangono invariati.

Di conseguenza, lo schema è già **normalizzato** e non presenta anomalie di aggiornamento, cancellazione o inserimento dovute a ridondanze. Pertanto, non è necessaria alcuna ulteriore normalizzazione.

MODELLO ER RISTRUTTURATO



MODELLO LOGICO

Utente(**email**, nome, cognome, luogo_nascita, anno_nascita, nickname, password)

Amministratore(**emailAmministratore**, codice)

Creatore(**emailCreatore**, affidabilita)

Finanziamento(**data**, **emailUtente**, **nomeProgetto**, importo, codiceReward)

Reward(**codice**, foto, descr, nomeProgetto)

Skill(**nome**)

Skill_Possesso(**emailUtente**, **nomeSkill**, livello)

Skill_Requisito(**nomeSkill**, **nomeProfilo**, **nomeProgetto**, livello)

Profilo(**nome**, **nomeProgetto**, numero_posizioni)

Candidatura(**id**, stato, accettata, nomeProfilo, nomeProgetto, emailUtente)

Commento(**id**, data, testo, emailUtente, nomeProgetto)

Risposta(**idCommento**, contenuto, emailCreatore)

Progetto(**nome**, data_inserimento, data_limite, descr, stato, budget_avvio, tipoProgetto, emailCreatore)

Foto_Progetto(**id**, descrizione, nomeProgetto)

Componente(**nome**, **nomeProgetto**, prezzo, descr, quantita)

VINCOLI DI INTEGRITA' REFERENZIALE

Amministratore.emailAmministratore -> Utente.email

Creatore.emailCreatore -> Utente.email

Finanziamento.emailUtente -> Utente.email

Finanziamento.nomeProgetto -> Progetto.nome

Finanziamento.codiceReward -> Reward.codice

Reward.nomeProgetto -> Progetto.nome

Skill_Possesso.emailUtente -> Utente.email

Skill_Possesso.nomeSkill -> Skill.nome

Skill_Requisito.nomeSkill -> Skill.nome

Skill_Requisito.nomeProfilo -> Profilo.nome

Skill_Requisito.nomeProgetto -> Profilo.nomeProgetto

Profilo.nomeProgetto -> Progetto.nome

Candidatura.nomeProfilo -> Profilo.nome

Candidatura.nomeProgetto -> Profilo.nomeProgetto

Candidatura.emailUtente -> Utente.email

Commento.emailUtente -> Utente.email

Commento.nomeProgetto -> Progetto.nome

Risposta.idCommento -> Commento.id

Risposta.emailCreatore -> Creatore.emailCreatore

Progetto.emailCreatore -> Creatore.emailCreatore

Foto_Progetto.nomeProgetto -> Progetto.nome

Componente.nomeProgetto -> Progetto.nome

MANUALE

[LOGIN](#)

Esistono tre tipologie di accesso al sistema: **utente normale**, **utente creatore** e **utente amministratore**.

Caso 1: Utente Normale

The image shows two identical login forms for 'BOSTARTER'. Each form has the title 'Accedi al tuo account'. The first form is empty, while the second has 'normal.user@email.com' in the 'Email' field and '*****' in the 'Password' field. Both forms feature a blue 'Login' button. Below the button, there are two links: 'Non hai un account? Registrati' in blue and 'Accedi come amministratore' in red.

Se un **utente normale** inserisce correttamente le proprie credenziali, verrà reindirizzato alla relativa schermata.

The screenshot shows the BOSTARTER home page for a normal user. The top navigation bar is blue with the 'BOSTARTER' logo on the left and a user profile section on the right containing a grey box with 'N', the text 'normalUser', and a red 'Logout' button. The left sidebar is dark blue with white links: 'Progetti disponibili', 'Le mie skill', 'Le mie candidature', 'I miei finanziamenti', and 'Visualizza statistiche'. The main content area is white and contains the message 'Benvenuto/a normalUser su Bostarter!' followed by 'Seleziona un'opzione dal menu per iniziare.' The footer is blue with the text 'BOSTARTER - Copyright ©, 2025'.

- In alto a destra comparirà una "N" all'interno di un rettangolo grigio, che indica l'accesso come **utente normale**.
- Sarà visibile il bottone rosso per effettuare il **logout**.
- Dalla schermata **HOME**, l'utente normale potrà:
 - Visualizzare i **progetti disponibili**.
 - Visualizzare e modificare le proprie **skill**.
 - Visualizzare le **candidature** inviate dall'utente.

- Visualizzare i **finanziamenti** effettuati dall'utente.
- Visualizzare le **statistiche**.

Caso 2: Utente Creatore

Se un **utente creatore** inserisce correttamente le proprie credenziali, accederà alla relativa schermata.

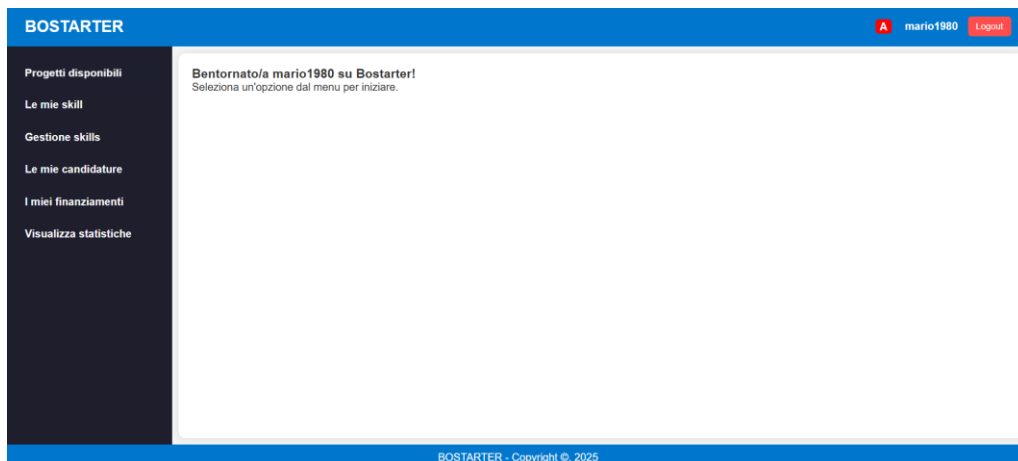
- In alto a destra comparirà una "C" all'interno di un rettangolo verde, che indica l'accesso come **utente creatore**.
- Sarà visibile il bottone rosso per effettuare il **logout**.
- Dalla schermata **HOME**, l'utente creatore potrà:
 - Visualizzare i **progetti disponibili**.
 - Creare un **nuovo progetto**.
 - Visualizzare i **progetti creati** dall'utente specifico.
 - Visualizzare e modificare le proprie **skill**.

- Visualizzare le **candidature** inviate dall'utente.
- Visualizzare i **finanziamenti** effettuati dall'utente.
- Visualizzare le **statistiche**.

Caso 3: Utente Amministratore

BOSTARTER - AMMINISTRAZIONE BOSTARTER - AMMINISTRAZIONE

Se un **utente amministratore** inserisce correttamente le proprie credenziali, verrà reindirizzato alla relativa schermata.

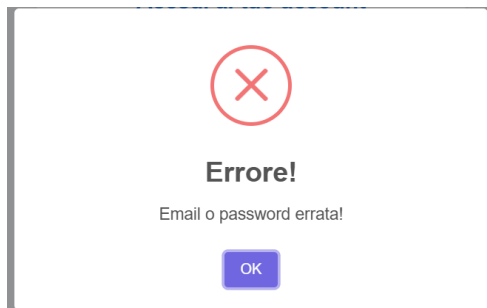


- In alto a destra comparirà una "A" all'interno di un rettangolo rosso, che indica l'accesso come **utente amministratore**.
- Sarà visibile il bottone rosso per effettuare il **logout**.
- Dalla schermata **HOME**, l'utente amministratore potrà:
 - Visualizzare i **progetti disponibili**.
 - Visualizzare e modificare le proprie **skill**.
 - Gestire le **competenze (skill)** del sistema.
 - Visualizzare le **candidature** inviate dall'utente.

- Visualizzare i **finanziamenti** effettuati dall'utente.
- Visualizzare le **statistiche**.

Errore nelle Credenziali

Se le credenziali (email o password) inserite sono errate, verrà visualizzato un messaggio di errore di questo tipo.



REGISTRAZIONE

Gli utenti possono registrarsi in base al ruolo desiderato. La registrazione avviene sempre cliccando sul link "**Non hai un account? Registrati**".

Registrazione Utente Normale

Per registrarsi come **utente normale**, **non deve** essere selezionata l'opzione per l'utente creatore.

Registrati

Nome

Cognome

Luogo di nascita

Anno di nascita

Email

Nickname

Password

Conferma Password

☐ Utente creatore

Registrati

[Hai già un account? Accedi](#)

Registrati

Nome

Cognome

Luogo di nascita

Anno di nascita

Email

Nickname

Password

Conferma Password

☐ Utente creatore

Registrati

[Hai già un account? Accedi](#)

Registrazione Utente Creatore

La registrazione avviene come per l'utente normale, ma **selezionando** la **checkbox** dedicata agli **utenti creatori**.

The image shows two side-by-side registration forms, both titled 'Registrati' in blue. The left form is empty, while the right form is filled with example data. Both forms include the following fields: Nome, Cognome, Luogo di nascita, Anno di nascita, Email, Nickname, Password, and Conferma Password. At the bottom of each form, there is a checked checkbox labeled 'Utente creatore', a blue 'Registrati' button, and a link 'Hai già un account? Accedi'.

Campo	Valore (Forma di destra)
Nome	Matteo
Cognome	Boscherini
Luogo di nascita	Forlì
Anno di nascita	2003
Email	matteo.boscherini17@gmail.com
Nickname	Mat17
Password	*****
Conferma Password	*****
Checkbox	☒ Utente creatore

Registrazione Utente Amministratore

La registrazione avviene da un modulo dedicato agli amministratori. Oltre ai dati standard, è richiesto un **codice di sicurezza** per completare la registrazione.

Registrati

Nome

Cognome

Luogo di nascita

Anno di nascita

Email

Nickname

Password

Conferma Password

Codice sicurezza

Registrati

[Hai già un account? Accedi](#)

Registrati

Nome

Cognome

Luogo di nascita

Anno di nascita

Email

Nickname

Password

Conferma Password

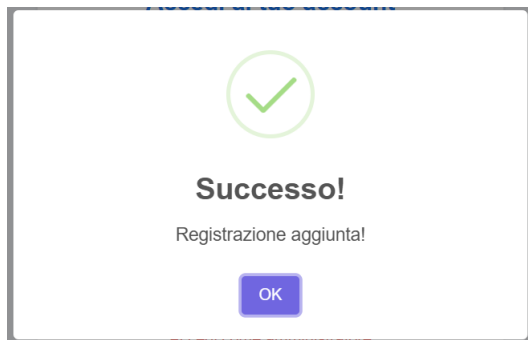
Codice sicurezza

Registrati

[Hai già un account? Accedi](#)

Conferma della Registrazione

Dopo aver compilato il form e cliccato sul bottone "**Registrati**", verrà mostrata una schermata di conferma.



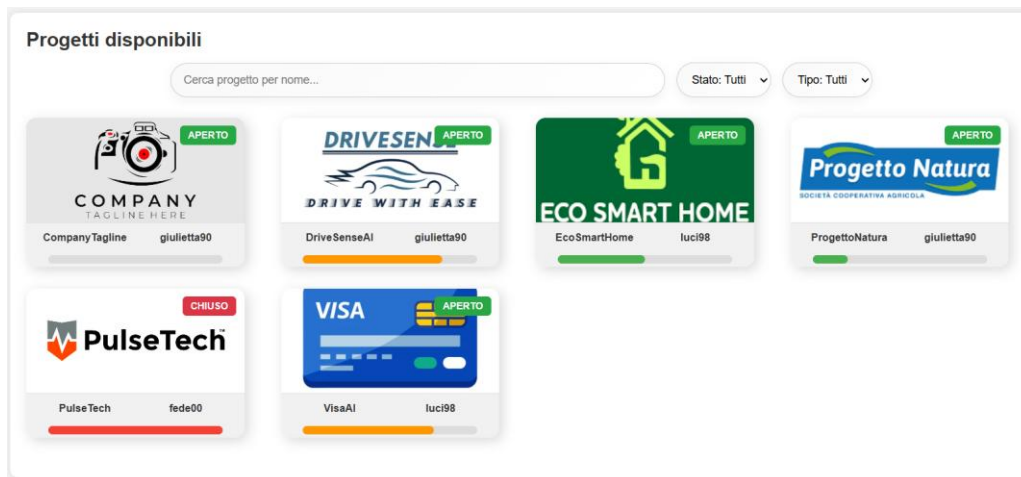
FUNZIONALITÀ PER GLI UTENTI

Le funzionalità variano in base al tipo di utente. Alcune sono comuni a tutti, altre sono esclusive per creatori e amministratori.

Visualizzazione dei Progetti

Tutti gli utenti possono accedere all'elenco dei **progetti disponibili**, sia **software** che **hardware**.

Cliccando su un progetto, verranno mostrate le seguenti informazioni:



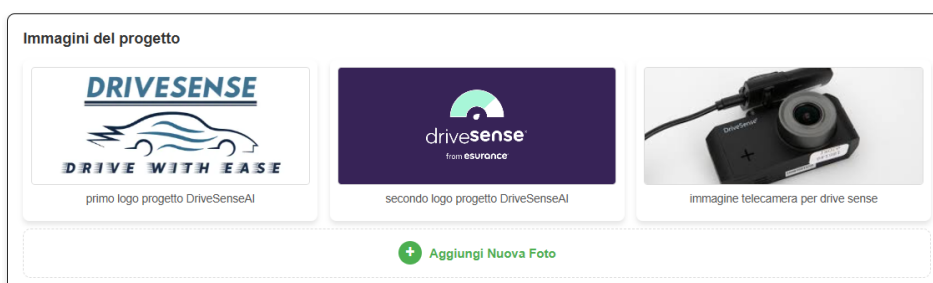
Cliccando su un **progetto** (ad esempio, *DriveSenseAI*), verranno mostrate le seguenti informazioni:

- **Descrizione generale del progetto**

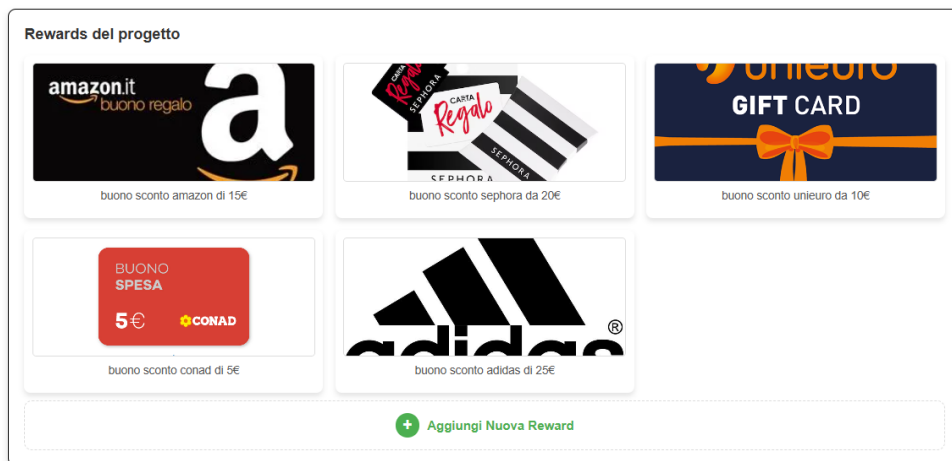
Dettagli del progetto DriveSenseAI

Informazioni
 Data di inserimento: 2025-03-25
 Data di fine: 2025-10-10
 Descrizione:
 Stato: aperto
 Tipo: Software
 Budget d'avvio: 500000.00 EUR
 Finanziamenti ricevuti: 400000.00 EUR

- **Galleria immagini del progetto** (solo il creatore può aggiungere/modificare immagini)



- **Reward disponibili del progetto** (modificabili solo dal creatore del progetto)



- **Profili richiesti con relative skill minime necessarie per candidarsi**

Profili richiesti

[Profili richiesti per questo progetto](#)

E cliccando sul link, verrà visualizzato questo modulo:

Profili disponibili - DriveSenseAI

Business Analyst

0 posizioni disponibili

[Vedi Dettagli](#)

Data Scientist

1 posizioni disponibili

[Vedi Dettagli](#)

- **Skill richieste per candidarsi cliccando il bottone “Vedi Dettagli”**

Skills richieste - Data Scientist

Di seguito sono riportate le skill, e il loro rispettivo livello, richieste per candidarsi al profilo

Nome	Livello Minimo
Machine Learning	3
Programmazione in Python	2

[Invia candidatura](#)

Se non si è l'utente creatore del progetto, è possibile candidarsi premendo **"Invia Candidatura"**.

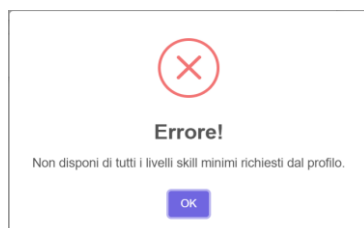


Dopo aver confermato con "**Sì, procedi**", l'esito potrà essere:

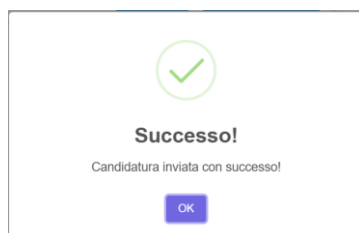
- Se l'utente avesse **già mandato** la candidatura per quel profilo.



- Se le **skill** fossero **insufficienti**, la candidatura non sarà accettata.



- Se le **skill** sono **adeguate o superiori**, la candidatura sarà valida e confermata.



- **Sezione commenti**

Commenti pubblicati

[Visualizza i commenti pubblicati per questo progetto](#)

E cliccando sul link, verrà visualizzato questo modulo.

Commenti - DriveSenseAI

lucif98 - 2025-03-25

Sono un sacco interessato a questa macchina, non vedo ora che sia il momento che sia pronta per poterla acquistare!

Rispondi

mario1980 - 2025-03-25

Sembra un progetto interessante!

Grazie per il supporto!

felice00 - 2025-03-25

Intelligenza artificiale sta cambiando il settore automobilistico e DriveSenseAI promette di portare la sicurezza stradale a un nuovo livello. Idea di un sistema avanzato di assistenza alla guida basato su machine learning è davvero interessante. Non vedo il momento di vedere come verrà implementato il rilevamento predittivo degli ostacoli!

Grazie Mario! Il nostro focus principale è proprio la sicurezza stradale. Il sistema sarà in grado di analizzare il comportamento degli altri veicoli e dei pedoni in tempo reale, per anticipare possibili pericoli e assistere il guidatore nelle decisioni critiche.

- **Componenti hardware** (solo per progetti hardware, modificabili solo dal creatore)

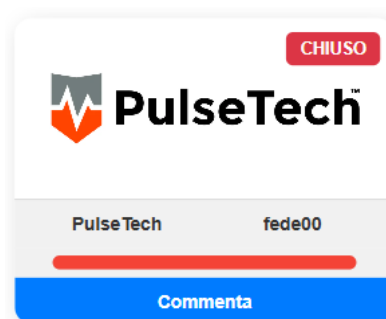
Componenti			
Nome	Prezzo (€)	Descrizione	Quantità
Collare GPS Intelligente	500.00	Dispositivo per tracciare il movimento degli animali selvatici	20
Drone di Monitoraggio Fauna	5000.00	Drone con telecamere AI per il monitoraggio degli animali	3
Sensore Acustico per Bracconaggio	1000.00	Microfono AI per rilevare spari e movimenti sospetti	5
Telecamera Notturna per Monitoraggio	1500.00	Telecamera a infrarossi per registrare attività notturna	6
 Aggiungi Nuova Componente			

INTERAZIONE CON I PROGETTI

Se il progetto è ancora **aperto** (non ha raggiunto il finanziamento completo) saranno disponibili due pulsanti: "**Commenta**" e "**Finanzia**". Se il progetto è **chiuso**, sarà disponibile solo il pulsante "Commenta".



(aperto)



(chiuso)

Inserimento di un Commento

1. Premere "**Commenta**" e si aprirà questo form.

Commenta il progetto DriveSenseAI

Testo del commento

Inserisci il testo del commento...

Invia Commento

2. Scrivere il **commento** nel form.

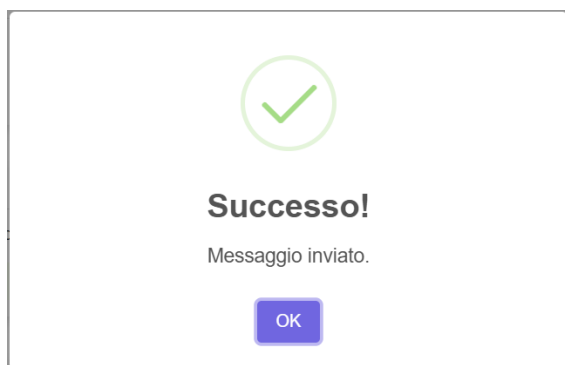
Commenta il progetto DriveSenseAI

Testo del commento

Bellissimo Progetto!

Invia Commento

3. Premere "**Invia commento**" e verrà visualizzato questo messaggio all'utente.



Finanziamento di un Progetto

1. Premere "**Finanzia**" e si aprirà questo form.

Finanzia il progetto ProgettoNatura

Avanzamento finanziamenti attuale
20.00% 10000 € / 50000 €

Importo (€)

Seleziona una reward cliccando sulla riga corrispondente

Codice	Descrizione	Foto
8	personalizzazione gratuita di un prodotto	
9	penna del progetto ProgettoNatura	
10	portachiavi del progetto ProgettoNatura	

[Invia Finanziamento](#)

2. Inserire l'**importo** desiderato e selezionare una **reward** dalla tabella.

Finanzia il progetto ProgettoNatura

Avanzamento finanziamenti attuale
20.00% 10000 € / 50000 €

Importo (€)
20000

Seleziona una reward cliccando sulla riga corrispondente

Codice	Descrizione	Foto
8	personalizzazione gratuita di un prodotto	
9	penna del progetto ProgettoNatura	
10	portachiavi del progetto ProgettoNatura	

[Invia Finanziamento](#)

3. Confermare premendo il bottone "**Si, procedi**" e se avviene ricevere conferma dell'avvenuto finanziamento.



Sei sicuro?

Vuoi finanziare veramente questo progetto?

[Si, procedi](#) [Annulla](#)



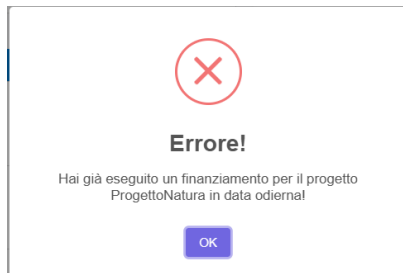
Successo!

Finanziamento eseguito.

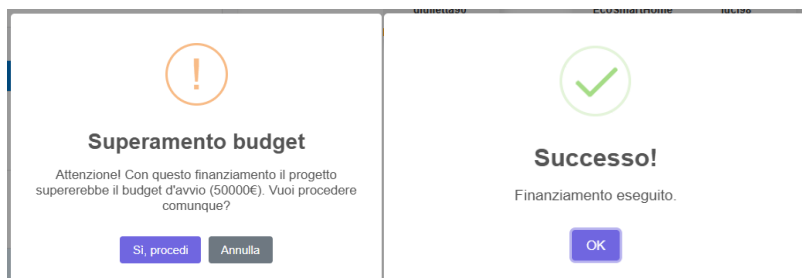
[OK](#)

Possibili Errori nel Finanziamento

Limitazione giornaliera: Non è possibile finanziare lo stesso progetto più di una volta al giorno dallo stesso utente e verrà mandato questo messaggio di output all'utente.

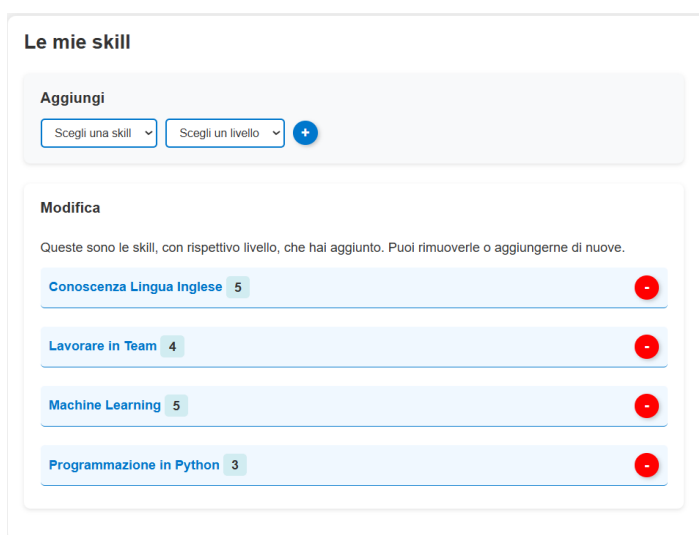


Superamento del budget totale: Se l'importo supera il budget richiesto dal progetto, verrà mostrato un avviso e se l'utente conferma comunque il finanziamento premendo il bottone "**Si, procedi**", l'operazione verrà eseguita mostrando la conferma dell'avvenuto finanziamento.



Gestione delle Skill

Gli utenti possono **visualizzare, aggiungere o rimuovere** le proprie skill.



Per **aggiungere** una **skill**, è sufficiente compilare il form e cliccare il pulsante blu "+" di aggiunta e confermare l'inserimento come nell'esempio qui sotto.

Le mie skill

Aggiungi

CyberSecurity

3

+

Modifica

Queste sono le skill, con rispettivo livello, che hai aggiunto. Puoi rimuoverle o aggiungerne di nuove.

Conoscenza Lingua Inglese

5

-

CyberSecurity

3

-

Lavorare in Team

4

-

Machine Learning

5

-

Programmazione in Python

3

-

Modifica

Queste sono le skill, con rispettivo livello, che hai aggiunto. Puoi rimuoverle o aggiungerne di nuove.

Conoscenza Lingua Inglese

5

-

CyberSecurity

3

-

Lavorare in Team

4

-

Machine Learning

5

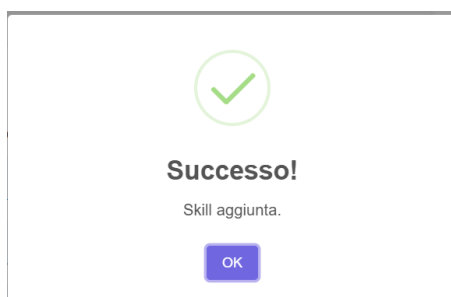
-

Programmazione in Python

3

-

Dopo l'inserimento, verrà visualizzato un messaggio di conferma all'utente.



Per **rimuovere** una **skill**, basta cliccare sul pulsante rosso "-" accanto alla skill desiderata da eliminare. Una volta confermata l'eliminazione, la skill scomparirà dall'elenco.

Le mie skill

Aggiungi

Scegli una skill

Scegli un livello

+

Modifica

Queste sono le skill, con rispettivo livello, che hai aggiunto. Puoi rimuoverle o aggiungerne di nuove.

Conoscenza Lingua Inglese

5

-

Lavorare in Team

4

-

Machine Learning

5

-

Programmazione in Python

3

-

Le mie skill

Aggiungi

Scegli una skill

Scegli un livello

+

Modifica

Queste sono le skill, con rispettivo livello, che hai aggiunto. Puoi rimuoverle o aggiungerne di nuove.

Lavorare in Team

4

-

Machine Learning

5

-

Programmazione in Python

3

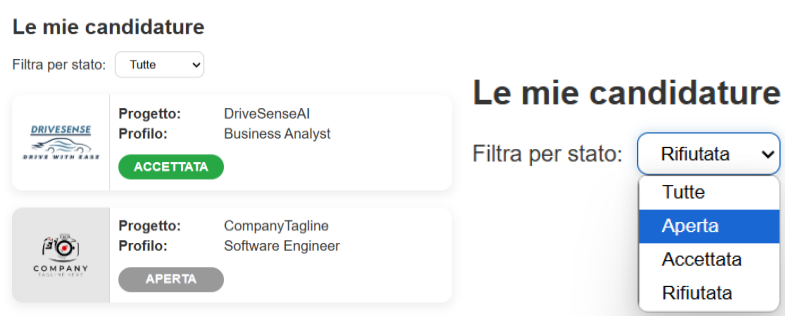
-

Gestione delle Candidature

L'utente ha accesso a una sezione dedicata alla gestione delle proprie **candidature**. Qui è possibile visualizzare tutte le candidature inviate per i diversi progetti. Le candidature sono suddivise nelle seguenti categorie:

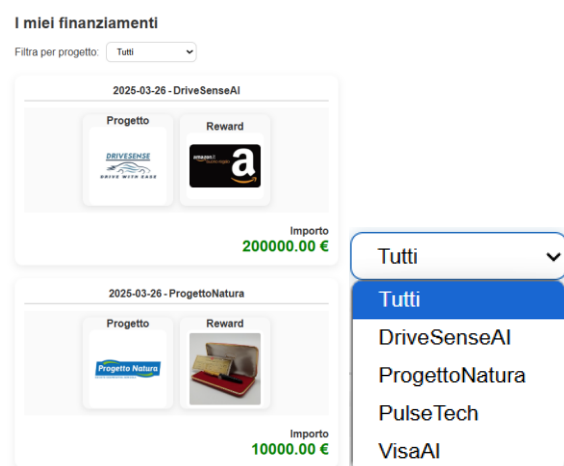
- **Aperte:** candidature ancora in attesa di valutazione da parte dell'utente creatore.
- **Accettate:** candidature approvate dall'utente creatore del progetto.
- **Rifiutate:** candidature non accettate dall'utente creatore del progetto.

L'interfaccia include un **filtro** che consente di visualizzare esclusivamente una delle categorie sopra elencate, facilitando la consultazione delle proprie candidature.



Gestione dei Finanziamenti

L'utente ha accesso a una sezione dedicata in cui può visualizzare tutti i **finanziamenti effettuati** per diversi progetti. L'interfaccia include un **filtro** che permette di selezionare e visualizzare esclusivamente i progetti in cui l'utente ha contribuito con un finanziamento.



Visualizzazione delle Statistiche

L'utente ha accesso a una sezione dedicata alle statistiche, che include: la classifica dei tre creatori più affidabili, ordinata in base al livello di affidabilità, la classifica dei tre progetti più vicini al completamento e la classifica dei tre utenti con il maggior contributo finanziario, mostrando i maggiori finanziatori del sistema nei vari progetti.

Statistiche

Classifica Creatori per Affidabilità

Nickname	Affidabilità (%)
fede00	100
luci98	100
giulietta90	67

Progetti Vicini al Completamento

Nome Progetto	Budget Mancante (€)	Budget Avvio (€)
ProgettoNatura	40000.00	50000.00
VisaAI	50000.00	200000.00
CompanyTagline	100000.00	100000.00

Classifica Finanziatori

Nickname	Totale Finanziamenti (€)
giulietta90	400000.00
mario1980	360000.00
fede00	350000.00

FUNZIONALITÀ UTENTE AMMINISTRATORE

L'utente amministratore, oltre alle funzionalità dell'utente normale, ha la possibilità di **gestire le competenze (skill) del sistema**.

Gestione Skills

Aggiungi



Modifica

Queste sono le skill. Puoi aggiungerne di nuove.

Conoscenza Lingua Inglese

CyberSecurity

Lavorare in Team

Machine Learning

Programmazione in Python

Per **aggiungere una nuova skill** basta compilare il form di aggiunta, inserire il nome della skill e premere il pulsante di conferma.

Gestione Skills

Aggiungi
 

Una volta aggiunta, la nuova skill sarà immediatamente visibile nell'elenco.

Gestione Skills

Aggiungi
 

Modifica
Queste sono le skill. Puoi aggiungerne di nuove.

Conoscenza Java

Conoscenza Lingua Inglese

CyberSecurity

Lavorare in Team

Machine Learning

Programmazione in Python

FUNZIONALITÀ UTENTE CREATORE

L'utente creatore, oltre alle funzionalità dell'utente normale, può **creare nuovi progetti e gestire quelli esistenti**.

Creazione di un Nuovo Progetto

Per avviare la creazione di un nuovo progetto, l'utente deve compilare il modulo iniziale e può creare un **progetto hardware**, che prevede l'inserimento di profili, immagini, reward e componenti hardware o un **progetto software**, che prevede l'inserimento di profili, immagini e reward.

Crea un nuovo progetto

Nome
nome

Data limite
gg/mm/aaaa

Descrizione
descrizione

Budget (€)
budget

Tipologia
☒ Hardware
☐ Software

Prosegui

Crea un nuovo progetto

Nome
SmartCoffee

Data limite
17/02/2026

Descrizione
Una macchinetta del caffè con l'AI

Budget (€)
150000

Tipologia
☐ Hardware
☒ Software

Prosegui

Dopo aver scelto il tipo di progetto, si procede con l'aggiunta dei **profili richiesti**:

1. Compilare il modulo dell'inserimento dei profili con le informazioni necessarie.

Inserimento profili

Nome Profilo
Manager

Posizioni disponibili
1

Skills richieste
Lavorare in Team 4

Aggiungi

Nessun profilo inserito

Prosegui

2. Cliccare su **"Aggiungi"** per confermare.

Manager 1 posizioni

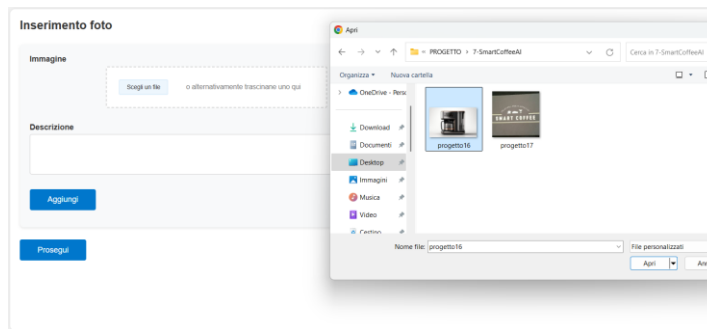
Skills

Skill	Livello
Lavorare in Team	4

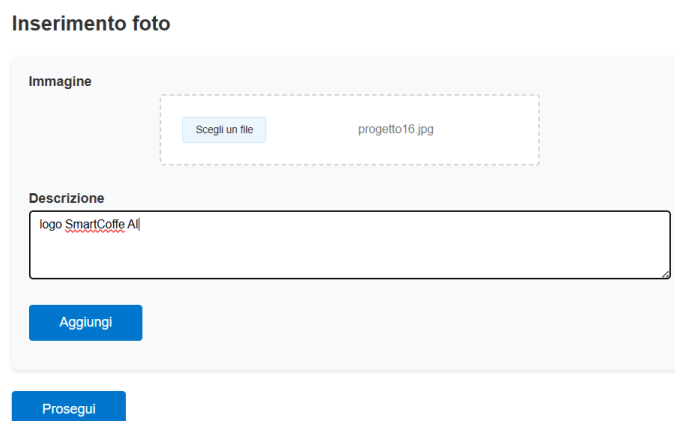
3. Una volta inseriti tutti i profili, premere **"Prosegui"**.

Per aggiungere delle **immagini al progetto**:

1. Caricare un'immagine.



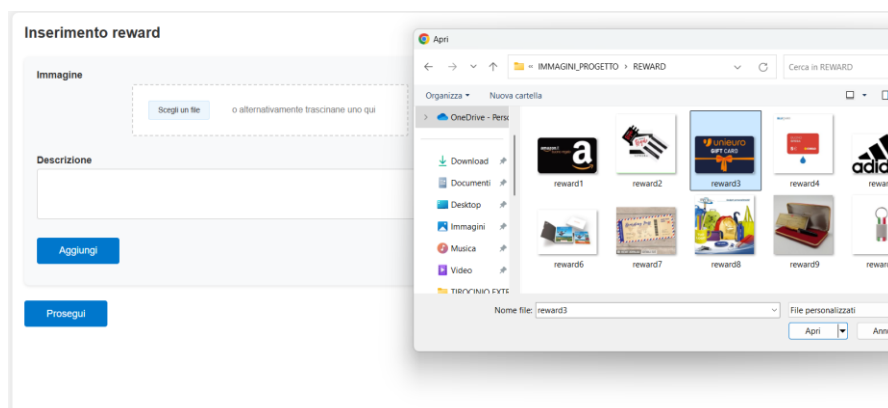
2. Inserire una descrizione e premere **"Aggiungi"** per confermare.



3. Dopo aver aggiunto tutte le immagini, cliccare su **"Prosegui"**.

Per aggiungere delle **reward** al **progetto**:

1. Selezionare un'immagine rappresentativa della reward.



2. Inserire una descrizione dettagliata e premere **"Aggiungi"** per confermare.

Inserimento reward

Immagine

Scegli un file reward3.png

Descrizione

buono sconto unieuro da 10€

Aggiungi

Prosegui

3. Dopo aver aggiunto tutte le reward, cliccare su "**Prosegui**".

Se il progetto è di tipo **hardware**, è possibile aggiungere anche **componenti hardware**:

1. Compilare il modulo con le informazioni richieste.

Inserimento componenti

Nome
Modulo Wi-Fi

Descrizione
Il modulo Wi-Fi per la macchinetta del caffè

Quantità
1

Prezzo (€)
100

Aggiungi componente

Prosegui

Nome	Descrizione	Quantità	Prezzo
Nessuna componente inserita			

2. Cliccare su "**Aggiungi componente**" per confermare.

Inserimento componenti

Nome
nome

Descrizione
descrizione

Quantità
quantità

Prezzo (€)
prezzo

Aggiungi componente

Prosegui

Nome	Descrizione	Quantità	Prezzo
Modulo Wi-Fi	Il modulo Wi-Fi per la macchinetta del caffè.	1	100

3. Dopo aver inserito tutte le componenti, premere "**Prosegui**".

Dopo aver completato tutte le **sezioni**, verrà mostrato un **riepilogo** del **progetto**.

Conferma dati

Informazioni Base del Progetto				
Nome	Data limite	Descrizione	Budget	Tipo
SmartCoffee	2026-02-17	Una macchinetta del caffè con l'AI	150000	software

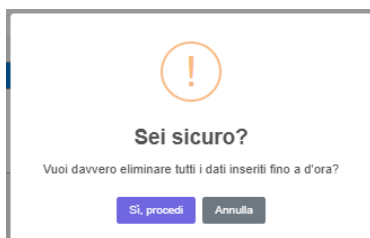
Profili Software Richiesti	
Manager	
1 posizioni	
Skills	
Skill	Livello
Lavorare in Team	4

Foto del progetto	
Descrizione	Foto
logo SmartCoffee	
logo 2 SmartCoffee	

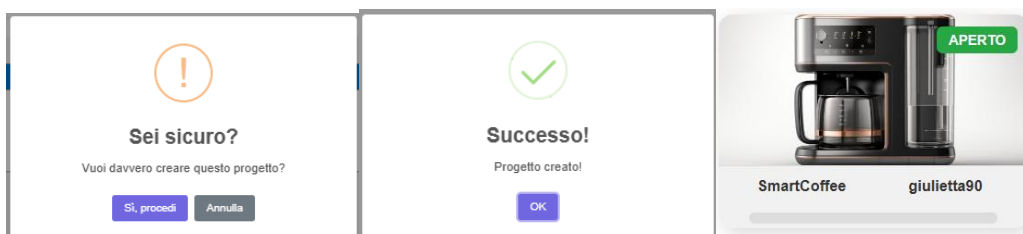
Rewards del Progetto	
Descrizione	Foto
buono sconto unieuro da 10€	

Annulla ed Elimina Conferma e Crea

- Se si clicca su "Annulla ed Elimina" e su "Sì, procedi" il progetto verrà cancellato.

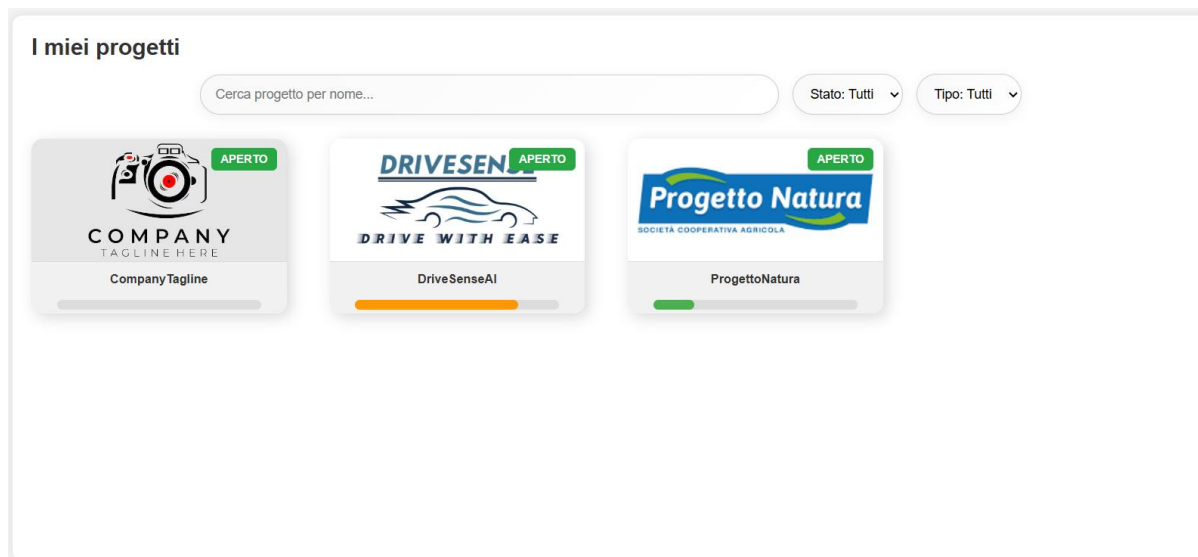


- Se si clicca su "Conferma e Crea" e su "Sì, procedi" il progetto verrà creato e verrà mostrata una schermata di conferma.

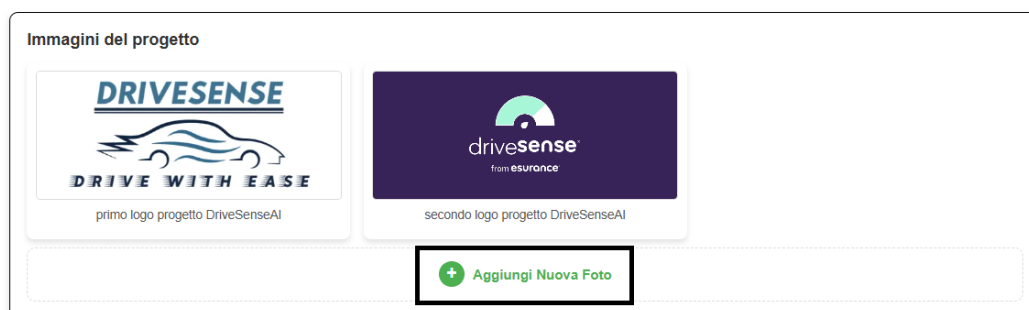


Gestione dei Progetti Creati

L'utente creatore può visualizzare tutti i **progetti** che ha creato e **modificarne** alcuni aspetti.



- **Modifica delle Immagini del Progetto**



- 1. Premere "Aggiungi Nuova Foto".**

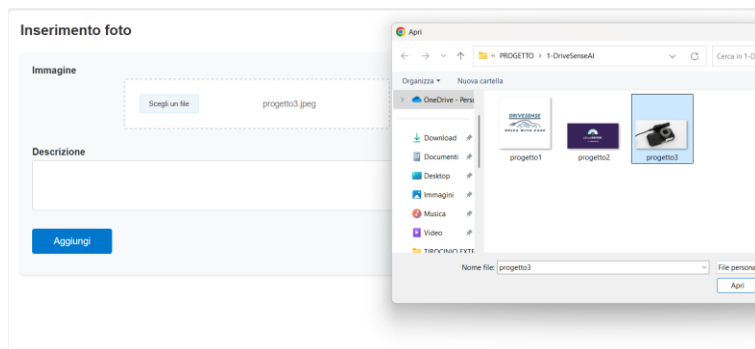
Inserimento foto

Immagine

o alternatively trascina uno qui

Descrizione

2. Selezionare un'immagine dal proprio dispositivo.



3. Inserire una descrizione.

Inserimento foto

Immagine

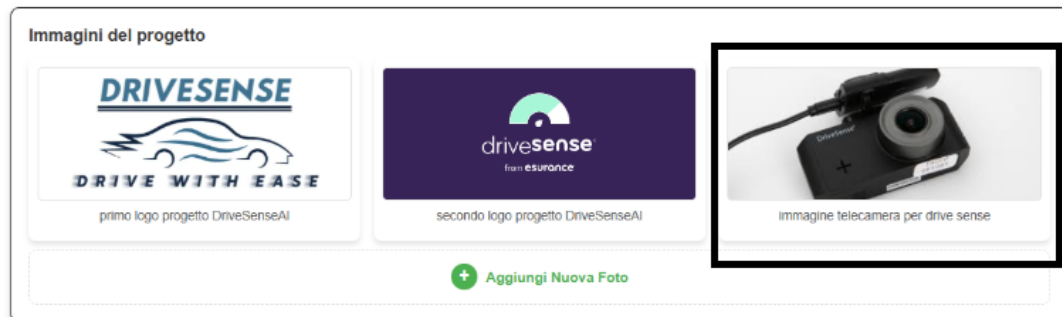
Scegli un file progetto3.jpeg

Descrizione

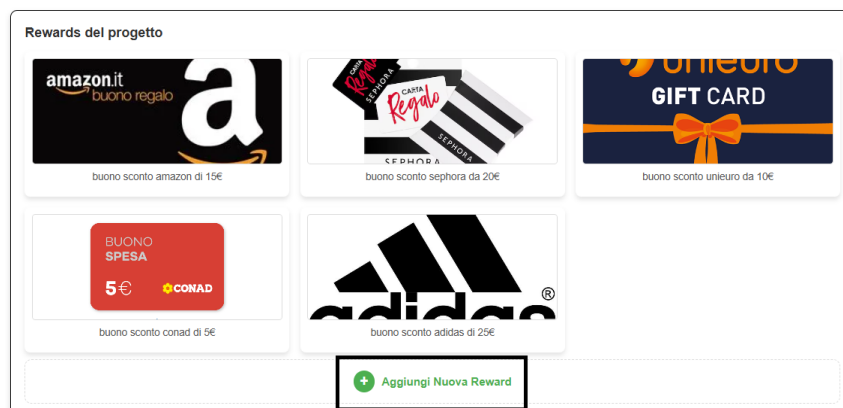
immagine telecamera per drive sense

Aggiungi

4. Cliccare su "Aggiungi" per confermare.



● Modifica delle Reward del Progetto



1. Premere "Aggiungi Nuova Reward".

Inserimento reward

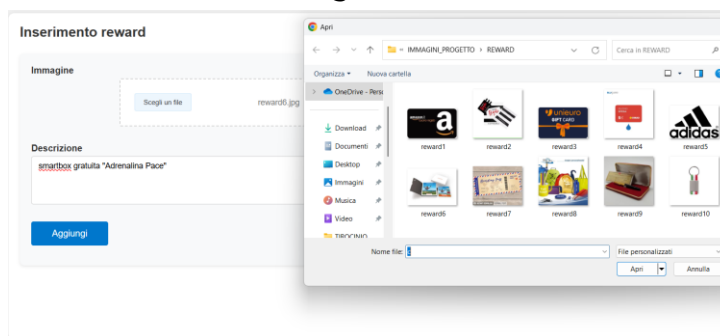
Immagine

Scegli un file o alternatively trascine uno qui

Descrizione

Aggiungi

2. Selezionare un'immagine.



3. Inserire una descrizione.

Inserimento reward

Immagine

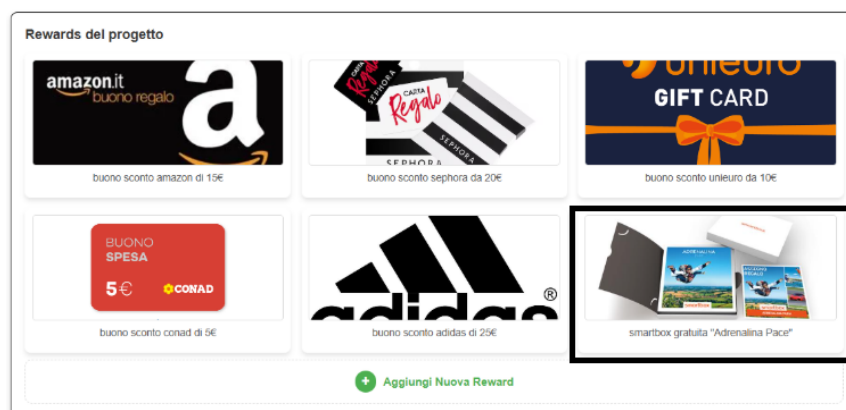
Scegli un file reward6.jpg

Descrizione

smartbox gratuita "Adrenalina Pace"

Aggiungi

4. Cliccare su "Aggiungi" per confermare.



- Gestione dei Profili Richiesti

Profili disponibili - DriveSenseAI

Business Analyst	0 posizioni disponibili	Vedi Dettagli	Visualizza Candidature
Data Scientist	1 posizioni disponibili	Vedi Dettagli	Visualizza Candidature
+ Aggiungi Nuovo Profilo			

1. Premere "Aggiungi Nuovo Profilo".

Inserimento profili

Nome Profilo

Posizioni disponibili

Skills richieste

Scegli una skill

Scegli un livello

Aggiungi

2. Inserire le informazioni richieste.

Inserimento profili

Nome Profilo

Manager

Posizioni disponibili

1

Skills richieste

Lavorare in Team 4

Conoscenza Lingua Inglese

4

Aggiungi

3. Aggiungere le **skill** necessarie tramite il pulsante "+".

Inserimento profili

Nome Profilo

Manager

Posizioni disponibili

1

Skills richieste

Lavorare in Team 4

Conoscenza Lingua Inglese 4

Scegli una skill

Scegli un livello

+

Aggiungi

4. Premere "Aggiungi" per confermare.

Profili disponibili - DriveSenseAI

Business Analyst

0 posizioni disponibili

Vedi Dettagli

Visualizza Candidature

Data Scientist

1 posizioni disponibili

Vedi Dettagli

Visualizza Candidature

Manager

1 posizioni disponibili

Vedi Dettagli

Visualizza Candidature

+ Aggiungi Nuovo Profilo

● Gestione delle Candidature ai Profili

L'utente creatore può **visualizzare** le **candidature** ricevute per i profili richiesti nel progetto e **accettarle** o **rifiutarle**.

Dettagli candidatura 5

Candidato

Nome e cognome:

Federica Verdi

Email:

federica.verdi@email.com

Anno di nascita

2000

Luogo di nascita

Napoli

Skills richieste

NOME	LIVELLO RICHIESTO	LIVELLO POSSEDUTO
Conoscenza Lingua Inglese	2	3
Machine Learning	3	4

Skills extra possedute dal candidato

NOME	LIVELLO
Lavorare in Team	4

Accetta

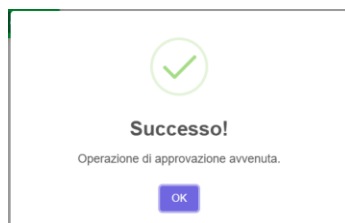
Rifiuta

Se si accetta una candidatura:

1. Cliccare sul pulsante verde **"Accetta"**.



2. Confermare premendo **"Sì, procedi"**.



3. La **candidatura** verrà aggiornata come **"Accettata"**.

Candidature

Profilo: Software Engineer

Progetto: CompanyTagline

Filtra per stato: Tutte ▼

Candidato: fede00

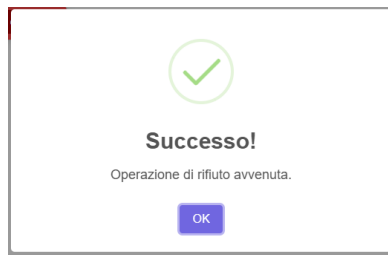
Vedi dettagli

Se si rifiuta una candidatura:

1. Cliccare sul pulsante rosso **"Rifiuta"**.



2. Confermare premendo **"Sì, procedi"**.



3. La **candidatura** verrà aggiornata come **"Rifiutata"**.

Candidature

Profilo: Software Engineer

Progetto: CompanyTagline

Filtra per stato: Tutte ▼

Candidato: fede00

Vedi dettagli

L'**utente creatore** può sempre **visualizzare** lo **stato** delle **candidature** e filtrare per lo stato di quest'ultimo.

Candidature

Profilo: Data Scientist

Progetto: DriveSenseAI

Filtra per stato: Tutte ▼

Candidato: luci98

Vedi dettagli

Candidato: mario1980

Vedi dettagli

- **Accettate** (con la possibilità di vedere i dettagli dell'utente).

Candidature

Profilo: Data Scientist

Progetto: DriveSenseAI

Filtra per stato: Accettata ▼

Candidato: mario1980

Vedi dettagli

Dettagli candidatura 2

Candidato	
Nome e cognome	Mario Rossi
Email	mario.rossi@email.com
Anno di nascita	1980
Lugaro di nascita	Roma

Skills richieste		
NOME	LIVELLO RICHIESTO	LIVELLO POSSEDUTO
Machine Learning	3	4
Programmazione in Python	2	5

Skills extra possedute dal candidato
Il candidato non possiede ulteriori skills

Questa candidatura è già stata esaminata

- **Rifiutate** (con la possibilità di vedere i dettagli dell'utente).

Candidature

Profilo: Data Scientist

Progetto: DriveSenseAI

Filtra per stato: Rifiutata

Candidato: **luci98**

Vedi dettagli

Dettagli candidatura 1

Candidato	
Nome e cognome	Lucia Giall
Email	luca.giall@gmail.com
Anno di nascita	1998
Luogo di nascita	Torino

Nome	Livello richiesto	Livello posseduto
Machine Learning	3	4
Programmazione in Python	2	4

Nome	Livello
CyberSecurity	2

Questa candidatura è già stata esaminata

- In attesa di risposta (in attesa di un'azione da parte del creatore).

Candidature

Profilo: Data Scientist

Progetto: DriveSenseAI

Filtra per stato: Aperta

Nessuna candidatura per il filtro selezionato

- Gestione dei Commenti

Commenti - DriveSenseAI

luci98 - 2025-03-25

Sono un sacco interessato a questa macchina, non vedo ora che sia il momento che sia pronta per poterla acquistare!

Rispondi

mario1980 - 2025-03-25

Sembra un progetto interessante!

Grazie per il supporto!

fedee00 - 2025-03-25

Intelligenza artificiale sta cambiando il settore automobilistico e DriveSenseAI promette di portare la sicurezza stradale a un nuovo livello. Idea di un sistema avanzato di assistenza alla guida basato su machine learning è davvero interessante. Non vedo il momento di vedere come verrà implementato il rilevamento predittivo degli ostacoli!

Grazie Mario! Il nostro focus principale è proprio la sicurezza stradale. Il sistema sarà in grado di analizzare il comportamento degli altri veicoli e dei pedoni in tempo reale, per anticipare possibili pericoli e assistere il guidatore nelle decisioni critiche.

1. Cliccare su "Rispondi" accanto a un commento.

Rispondi al commento

luci98 - 2025-03-25

Sono un sacco interessato a questa macchina, non vedo ora che sia il momento che sia pronta per poterla acquistare!

Scrivi la tua risposta

Invia messaggio

2. Scrivere la **risposta** nel modulo che appare.

Rispondi al commento

luc188 - 2025-03-25

Sono un sacco interessato a questa macchina, non vedo ora che sia il momento che sia pronta per poterla acquistare!

Sono molto contenti!

Invia messaggio

3. Premere **"Invia messaggio"** per confermare.

Commenti - DriveSenseAI

luc188 - 2025-03-25

Sono un sacco interessato a questa macchina, non vedo ora che sia il momento che sia pronta per poterla acquistare!

Sono molto contenti!

mario1880 - 2025-03-25

Sembra un progetto interessante!

Grazie per il supporto!

rodo00 - 2025-03-25

Intelligenza artificiale sta cambiando il settore automobilistico e DriveSenseAI promette di portare la sicurezza stradale a un nuovo livello. Idea di un sistema avanzato di assistenza alla guida basato su machine learning è davvero interessante. Non vedo il momento di vedere come verrà implementato il rilevamento predittivo degli ostacoli!

Grazie Mario! Il nostro focus principale è proprio la sicurezza stradale. Il sistema sarà in grado di analizzare il comportamento degli altri veicoli e dei pedoni in tempo reale, per anticipare possibili pericoli e assistere il guidatore nelle decisioni critiche.

● Gestione delle Componenti Hardware (Solo per progetti hardware)

Componenti			
Nome	Prezzo (€)	Descrizione	Quantità
Collare GPS Intelligente	500.00	Dispositivo per tracciare il movimento degli animali selvatici	20
Drone di Monitoraggio Fauna	5000.00	Drone con telecamere AI per il monitoraggio degli animali	3
Sensore Acustico per Bracconaggio	1000.00	Microfono AI per rilevare spari e movimenti sospetti	5
+ Aggiungi Nuova Componente			

1. Premere **"Aggiungi Nuova Componente"**.

Inserimento componenti

Nome

nome

Descrizione

descrizione

Quantità

quantità

Prezzo (€)

prezzo

Aggiungi componente

2. Compilare il modulo con le informazioni richieste.

Inserimento componenti

Nome

Telecamera Notturna per Monitoraggio

Descrizione

Telecamera a infrarossi per registrare attività notturna

Quantità

6

Prezzo (€)

1500

Aggiungi componente

3. Premere "Aggiungi componente" per confermare.

Componenti			
Nome	Prezzo (€)	Descrizione	Quantità
Collare GPS Intelligente	500.00	Dispositivo per tracciare il movimento degli animali selvatici	20
Drone di Monitoraggio Fauna	5000.00	Drone con telecamere AI per il monitoraggio degli animali	3
Sensore Acustico per Bracconaggio	1000.00	Microfono AI per rilevare spari e movimenti sospetti	5
Telecamera Notturna per Monitoraggio	1500.00	Telecamera a infrarossi per registrare attività notturna	6

+ Aggiungi Nuova Componente

CODICE SQL

```
#rimuovo il database "BOTSTARTER" se esiste già o creo il database "BOTSTARTER" se non esiste già
DROP DATABASE IF EXISTS BOSTARTER;
CREATE database IF NOT exists BOSTARTER;
USE BOSTARTER;
-- Creazione delle tabelle
CREATE TABLE Utente (
  email VARCHAR(255) PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  cognome VARCHAR(100) NOT NULL,
  luogo_nascita VARCHAR(100) NOT NULL,
  anno_nascita INT NOT NULL,
  nickname VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL
) ENGINE=INNODB;
CREATE TABLE Amministratore (
  emailAmministratore VARCHAR(255) PRIMARY KEY,
  codice INT NOT NULL,
  FOREIGN KEY (emailAmministratore) REFERENCES Utente(email)
) ENGINE=INNODB;
CREATE TABLE Creatore (
  emailCreatore VARCHAR(255) PRIMARY KEY,
  affidabilita INT DEFAULT 0,
  FOREIGN KEY (emailCreatore) REFERENCES Utente(email)
) ENGINE=INNODB;
CREATE TABLE Progetto (
  nome VARCHAR(255) PRIMARY KEY,
  data_inserimento DATE NOT NULL,
  data_limite DATE NOT NULL,
  descr TEXT NOT NULL,
  stato ENUM('aperto', 'chiuso') DEFAULT 'aperto',
  budget_avvio DECIMAL(10,2) NOT NULL,
  tipoProgetto ENUM('Hardware', 'Software') NOT NULL,
  emailCreatore VARCHAR(255),
  FOREIGN KEY (emailCreatore) REFERENCES Creatore(emailCreatore)
) ENGINE=INNODB;
CREATE TABLE Reward (
  codice INT PRIMARY KEY auto_increment,
  urlFoto VARCHAR(100) NOT NULL,
  descr TEXT NOT NULL,
  nomeProgetto VARCHAR(255),
```

```

        FOREIGN KEY (nomeProgetto) REFERENCES Progetto(nome)
    ) ENGINE=INNODB;
CREATE TABLE Finanziamento (
    data DATE,
    emailUtente VARCHAR(255),
    nomeProgetto VARCHAR(255),
    importo DECIMAL(10,2) NOT NULL,
    codiceReward INT,
    PRIMARY KEY (data, emailUtente, nomeProgetto),
    FOREIGN KEY (emailUtente) REFERENCES Utente(email),
    FOREIGN KEY (nomeProgetto) REFERENCES Progetto(nome),
    FOREIGN KEY (codiceReward) REFERENCES Reward(codice)
) ENGINE=INNODB;
CREATE TABLE Skill (
    nome VARCHAR(100) PRIMARY KEY
) ENGINE=INNODB;
CREATE TABLE Profilo (
    nome VARCHAR(100),
    nomeProgetto VARCHAR(255),
    numero_posizioni INT NOT NULL,
    PRIMARY KEY (nome, nomeProgetto),
    FOREIGN KEY (nomeProgetto) REFERENCES Progetto(nome)
) ENGINE=INNODB;
CREATE TABLE Skill_Possesso (
    emailUtente VARCHAR(255),
    nomeSkill VARCHAR(100),
    livello INT CHECK (livello BETWEEN 0 AND 5) NOT NULL,
    PRIMARY KEY (emailUtente, nomeSkill),
    FOREIGN KEY (emailUtente) REFERENCES Utente(email),
    FOREIGN KEY (nomeSkill) REFERENCES Skill(nome) ON DELETE CASCADE
) ENGINE=INNODB;
CREATE TABLE Skill_Requisito (
    nomeSkill VARCHAR(100),
    nomeProfilo VARCHAR(100),
    nomeProgetto VARCHAR(255),
    livello INT CHECK (livello BETWEEN 0 AND 5) NOT NULL,
    PRIMARY KEY (nomeSkill, nomeProfilo, nomeProgetto),
    FOREIGN KEY (nomeSkill) REFERENCES Skill(nome) ON DELETE CASCADE,
    FOREIGN KEY (nomeProfilo, nomeProgetto) REFERENCES Profilo(nome, nomeProgetto)
) ENGINE=INNODB;
CREATE TABLE Candidatura (
    id INT PRIMARY KEY AUTO_INCREMENT,
    stato ENUM('aperta', 'chiusa') default 'aperta',
    accettata BOOLEAN default false,
    nomeProfilo VARCHAR(100),
    nomeProgetto VARCHAR(255),
    emailUtente VARCHAR(255),
    FOREIGN KEY (nomeProfilo, nomeProgetto) REFERENCES Profilo(nome, nomeProgetto),
    FOREIGN KEY (emailUtente) REFERENCES Utente(email)
) ENGINE=INNODB;
CREATE TABLE Commento (
    id INT PRIMARY KEY AUTO_INCREMENT,
    data DATE NOT NULL,
    testo TEXT NOT NULL,
    emailUtente VARCHAR(255),
    nomeProgetto VARCHAR(255),
    FOREIGN KEY (emailUtente) REFERENCES Utente(email),
    FOREIGN KEY (nomeProgetto) REFERENCES Progetto(nome)
) ENGINE=INNODB;
CREATE TABLE Risposta (
    idCommento INT PRIMARY KEY,
    contenuto TEXT NOT NULL,
    emailCreatore VARCHAR(255),
    FOREIGN KEY (idCommento) REFERENCES Commento(id),
    FOREIGN KEY (emailCreatore) REFERENCES Creatore(emailCreatore)
) ENGINE=INNODB;
CREATE TABLE Foto_Progetto (
    id INT PRIMARY KEY AUTO_INCREMENT,
    descrizione TEXT NOT NULL,
    nomeProgetto VARCHAR(255),
    urlImmagine VARCHAR(100) NOT NULL,
    FOREIGN KEY (nomeProgetto) REFERENCES Progetto(nome)
) ENGINE=INNODB;
CREATE TABLE Componente (
    nome VARCHAR(255),
    nomeProgetto VARCHAR(255),
    prezzo DECIMAL(10,2) NOT NULL,
    descr TEXT NOT NULL,
    quantita INT CHECK (quantita > 0) NOT NULL,
    PRIMARY KEY (nome, nomeProgetto),
    FOREIGN KEY (nomeProgetto) REFERENCES Progetto(nome)
) ENGINE=INNODB;

```

```

-- Il trigger per cambiare lo stato del progetto da "aperto" a "chiuso"
DELIMITER $
CREATE TRIGGER CambioStatoProgetto AFTER INSERT ON Finanziamento FOR EACH ROW
BEGIN
    declare budgetRaggiunto decimal(10,2);
    declare budgetAvvio decimal(10,2);
    set budgetRaggiunto = (SELECT SUM(importo) FROM Finanziamento WHERE nomeProgetto =
NEW.nomeprogetto);
    set budgetAvvio = (SELECT budget_avvio FROM Progetto WHERE nome = NEW.nomeProgetto);
    if(budgetraggiunto >= budgetAvvio) then
        UPDATE Progetto SET stato = 'chiuso' WHERE nome = NEW.nomeProgetto;
    end if;
END;
$ DELIMITER ;
-- Il trigger che permette di aggiornare l'affidabilità dopo ogni volta che l'utente crea un
progetto
DELIMITER $
CREATE TRIGGER AggiornaAffidabilitaProgetto AFTER INSERT ON Progetto FOR EACH ROW
BEGIN
    declare numProgetti INT;
    declare numProgettiFinanziati INT;
    -- Il numero totale di progetti creati dall'utente creatore
    set numProgetti = (SELECT COUNT(*) FROM Progetto WHERE emailCreatore = NEW.emailCreatore);
    -- Il numero di progetti dell'utente creatore che hanno ricevuto almeno un finanziamento
    set numProgettiFinanziati = (SELECT COUNT(DISTINCT nomeProgetto) FROM Finanziamento WHERE
nomeProgetto IN (SELECT nome FROM Progetto WHERE emailCreatore = NEW.emailCreatore));
    -- Se l'utente ha almeno un progetto creato, aggiorniamo l'affidabilità dopo l'inserimento del
progetto
    IF(numProgetti > 0) THEN
        UPDATE Creatore SET affidabilita = (numProgettiFinanziati / numProgetti * 100) WHERE
emailCreatore = NEW.emailCreatore;
    END IF;
END;
$ DELIMITER ;
-- Il trigger che permette di aggiornare l'affidabilità dopo ogni volta il progetto riceve un
finanziamento
DELIMITER $
CREATE TRIGGER AggiornaAffidabilitaFinanziamento AFTER INSERT ON Finanziamento FOR EACH ROW
BEGIN
    declare emailCreatoreProgetto VARCHAR(255);
    declare numProgetti INT;
    declare numProgettiFinanziati INT;
    -- La email del creatore del progetto finanziato
    set emailCreatoreProgetto = (SELECT emailCreatore FROM Progetto WHERE nome = NEW.nomeProgetto);
    -- Il numero totale di progetti creati dall'utente creatore
    set numProgetti = (SELECT COUNT(*) FROM Progetto WHERE emailCreatore = emailCreatoreProgetto);
    -- Il numero di progetti dell'utente creatore che hanno ricevuto almeno un finanziamento
    set numProgettiFinanziati = (SELECT COUNT(DISTINCT nomeProgetto) FROM Finanziamento WHERE
nomeProgetto IN (SELECT nome FROM Progetto WHERE emailCreatore = emailCreatoreProgetto));
    -- Se l'utente ha almeno un progetto creato, aggiorniamo l'affidabilità dopo l'inserimento del
finanziamento
    IF(numProgetti > 0) THEN
        UPDATE Creatore SET affidabilita = (numProgettiFinanziati / numProgetti * 100) WHERE
emailCreatore = emailCreatoreProgetto;
    END IF;
END;
$ DELIMITER ;
DELIMITER $
CREATE TRIGGER DecrementaNumeroPosizioni AFTER UPDATE ON Candidatura FOR EACH ROW
BEGIN
    UPDATE Profilo SET numero_posizioni = numero_posizioni - 1 WHERE
nome = NEW.nomeProfilo and
nomeProgetto = NEW.nomeProgetto and
NEW.accettata = 1;
END ;
$ DELIMITER ;
DELIMITER $
CREATE EVENT CambiaStatoProgetto ON SCHEDULE EVERY 1 DAY STARTS CURRENT_DATE ()
DO
BEGIN
    UPDATE Progetto SET stato = 'chiuso' WHERE data_limite <= CURDATE() AND stato <> 'chiuso';
END;
$ DELIMITER ;
-- OPERAZIONI RIGARDANTI GLI UTENTI:
-- Autenticazione Utente normale: se non viene trovato l'utente ritorna 0, se la psw è errata
ritorna 1, se è corretta torna 2
DELIMITER $
CREATE PROCEDURE AutenticazioneNormale(IN emailI VARCHAR(255), IN passwordI VARCHAR(255), OUT esito
INT)
BEGIN
    declare esisteUtente boolean;
    declare passwordCorretta boolean;

```

```

        set esisteUtente = emailI IN (SELECT email FROM Utente) AND (emailI NOT IN (SELECT
emailAmministratore FROM Amministratore));
        set passwordCorretta = (SELECT count(*) FROM Utente WHERE email = emailI AND password =
MD5(passwordI)) > 0;
        if (esisteUtente AND passwordCorretta) then
            set esito = 1;
        else
            set esito = 0;
        end if;
END;
$ DELIMITER ;
-- Autenticazione Amministratore: se non viene trovato l'utente ritorna 0, se la psw è errata
ritorna 1, se è corretta torna ma il codice errato torna 2, mentre se tutto giusto 3
DELIMITER $
CREATE PROCEDURE AutenticazioneAmministratore(IN emailI VARCHAR(255), IN passwordI VARCHAR(255), IN
codiceI VARCHAR(50), OUT esito INT)
BEGIN
    declare esisteUtente boolean;
    declare passwordCorretta boolean;
    declare codiceCorretto boolean;
    set codiceI = CAST(codiceI AS UNSIGNED);
    set esisteUtente = emailI IN (SELECT emailAmministratore FROM Amministratore);
    set passwordCorretta = (SELECT count(*) FROM Utente WHERE email = emailI and MD5(passwordI) =
password) > 0;
    set codiceCorretto = (SELECT count(*) FROM Amministratore WHERE emailAmministratore = emailI AND
codice = codiceI);
    if ((NOT esisteUtente) OR (NOT passwordCorretta) OR (NOT codiceCorretto)) then
        set esito = 0;
    else
        set esito = 1;
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE RegistrazioneNormale(
    IN emailI VARCHAR(255),
    IN passwordI VARCHAR(255),
    IN nomeI VARCHAR(100),
    IN cognomeI VARCHAR(100),
    IN luogo_nascitaI VARCHAR(100),
    IN anno_nascitaI VARCHAR(4),
    IN nicknameI VARCHAR(50),
    OUT esito INT)
BEGIN
    declare esisteUtente boolean;
    declare esisteNickname boolean;
    set esisteUtente = emailI IN (SELECT email from Utente);
    set esisteNickname = nicknameI IN (SELECT nickname from Utente);
    if((NOT esisteUtente) AND (NOT esisteNickname)) then
        set anno_nascitaI = CAST(anno_nascitaI AS UNSIGNED);
        INSERT INTO Utente VALUES (emailI, nomeI, cognomeI, luogo_nascitaI, anno_nascitaI,
nicknameI, MD5(passwordI));
        set esito = 1; -- utente registrato
    else
        set esito = 0; -- utente non registrato
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE RegistrazioneCreatore(
    IN emailI VARCHAR(255),
    IN passwordI VARCHAR(255),
    IN nomeI VARCHAR(100),
    IN cognomeI VARCHAR(100),
    IN luogo_nascitaI VARCHAR(100),
    IN anno_nascitaI VARCHAR(4),
    IN nicknameI VARCHAR(50),
    OUT esito INT)
BEGIN
    CALL RegistrazioneNormale(emailI, passwordI, nomeI, cognomeI, luogo_nascitaI, anno_nascitaI,
nicknameI, @esitoNormale);
    if ( @esitoNormale = 0) then
        set esito = 0; -- creatore non registrato
    else
        set esito = 1; -- creatore registrato
        INSERT INTO Creatore (emailCreatore) VALUES (emailI);
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE RegistrazioneAmministratore(
    IN emailI VARCHAR(255),

```

```

    IN passwordI VARCHAR(255),
    IN nomeI VARCHAR(100),
    IN cognomeI VARCHAR(100),
    IN luogo_nascitaI VARCHAR(100),
    IN anno_nascitaI VARCHAR(4),
    IN nicknameI VARCHAR(50),
    IN codiceI VARCHAR(50),
    OUT esito INT)
BEGIN
    CALL RegistrazioneNormale(emailI, passwordI, nomeI, cognomeI, luogo_nascitaI, anno_nascitaI,
nicknameI, @esitoNormale);
    if ( @esitoNormale = 0) then
        set esito = 0; -- amministratore non registrato
    else
        set esito = 1; -- amministratore registrato
        set codiceI = (CAST(codiceI AS UNSIGNED));
        INSERT INTO Amministratore VALUES (emailI, codiceI);
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE InserimentoSkillCurriculum (IN emailI VARCHAR(255), IN nomeSkillI VARCHAR(255), IN
livelloI CHAR, OUT esito INT)
BEGIN
    declare emailCorretta boolean;
    declare nomeCorretto boolean;
    declare livelloCorretto boolean;
    set emailCorretta = emailI IN (SELECT email from Utente);
    set nomeCorretto = nomeSkillI IN (SELECT nome from Skill);
    set livelloCorretto = (livelloI REGEXP '^[0-5]$');
    if (emailCorretta and nomeCorretto and livelloCorretto) then
        set livelloI = (CAST(livelloI AS UNSIGNED));
        INSERT INTO Skill_Possesso VALUES (emailI, nomeSkillI, livelloI);
        set esito = 1; -- inserimento con successo
    else
        set esito = 0; -- inserimento con insuccesso
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE RimozioneSkillCurriculum(IN emailI VARCHAR(255), IN nomeSkillI VARCHAR(255), OUT
esito INT)
BEGIN
    declare esisteSkill boolean;
    set esisteSkill = nomeSkillI IN (SELECT nomeSkill from Skill_Possesso WHERE emailUtente = emailI
AND nomeSkill = nomeSkillI);
    if (esisteSkill) then
        DELETE FROM Skill_Possesso WHERE emailUtente = emailI AND nomeSkill = nomeSkillI;
        set esito = 1;
    else
        set esito = 0;
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE VisualizzaProgettiAperti()
BEGIN
    SELECT * FROM Progetto WHERE stato = "aperto" ORDER BY data_inserimento DESC;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE InserimentoFinanziamento(IN nomeProgettoI VARCHAR(255), IN importoI VARCHAR(50), IN
emailI VARCHAR(255), OUT esito INT)
BEGIN
    declare progettoValido boolean;
    declare emailCorretta boolean;
    -- Valido se il finanziamento che sta venendo fatto è in una data diversa dagli altri
finanziamenti fatti dallo stesso utente
    declare utenteValido boolean;
    declare importoValido boolean;
    declare importoSuperioreBudget boolean;
    declare budgetCorrente decimal;
    declare budgetAvvio decimal;
    -- Controllo se ha finanziamenti in quanto, se cerco di inserire uno in un progetto che non ne
ha
    declare haFinanziamenti boolean;
    declare importoDecimal decimal;
    set progettoValido = nomeProgettoI IN (SELECT nome FROM Progetto WHERE stato = 'aperto');
    set emailCorretta = emailI IN (SELECT email FROM Utente);
    set utenteValido = emailI NOT IN (SELECT emailUtente FROM Finanziamento WHERE nomeProgetto =
nomeProgettoI AND data = current_date());
    set budgetAvvio = (SELECT budget_avvio FROM Progetto WHERE nome = nomeProgettoI);

```

```

set haFinanziamenti = nomeProgettoI IN (SELECT nomeProgetto FROM Finanziamento);
if(haFinanziamenti) then
    set budgetCorrente = (SELECT SUM(importo) FROM Finanziamento WHERE nomeProgetto =
nomeProgettoI);
else
    set budgetCorrente = 0.0;
end if;
SET importoDecimal = CAST(importoI AS DECIMAL(10,2));
-- Controllo la validità dell'importo (deve essere maggiore di 0)
SET importoValido = importoDecimal > 0;
if(NOT(progettoValido AND emailCorretta AND importoValido)) then
    -- errore: progetto o utente non trovati o utente ha già eseguito finanziamento o l'importo
    inserito non è valido
    set esito = 0;
else
    -- errore: avviso l'utente che ha già fatto un finanziamento quel giorno per quel progetto
    if(NOT(utenteValido)) then
        set esito = 1;
    else
        -- è andato tutto a buon fine, permetto l'inserimento del finanziamento
        set esito = 2;
        INSERT INTO Finanziamento (data, emailUtente, nomeProgetto, importo) VALUES
(current_date(), emailI, nomeProgettoI, CAST(importoI AS DECIMAL(10,2)));
    end if;
end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE SceltaReward (IN codiceRewardI VARCHAR(50), IN emailUtenteI VARCHAR(255), IN
nomeProgettoI VARCHAR(255), OUT esito INT)
BEGIN
    declare rewardCorretta boolean;
    set rewardCorretta = CAST(codiceRewardI AS UNSIGNED) IN (SELECT codice FROM Reward WHERE
nomeProgetto = nomeProgettoI);
    if (codiceRewardI REGEXP '^[0-9]+$' and rewardCorretta) then
        set esito = 1;
        UPDATE Finanziamento SET codiceReward = CAST(codiceRewardI AS UNSIGNED) WHERE emailUtente =
emailUtenteI AND nomeProgetto = nomeProgettoI AND data = current_date();
    else
        set esito = 0;
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE CommentaProgetto(IN nomeProgettoI VARCHAR(255), IN emailAutoreI VARCHAR(255), IN
testoI TEXT, OUT esito INT)
BEGIN
    declare progettoEsistente boolean;
    declare autoreEsistente boolean;
    declare testoNonVuoto boolean;
    declare nonCreatore boolean;
    set progettoEsistente = nomeProgettoI IN (SELECT nome FROM Progetto WHERE nome = nomeProgettoI);
    set autoreEsistente = emailAutoreI IN (SELECT email FROM Utente WHERE email = emailAutoreI);
    set testoNonVuoto = LENGTH(trim(testoI)) > 0;
    -- utente creatore non può commentare il suo progetto, solo rispondere
    set nonCreatore = emailAutoreI NOT IN (SELECT emailCreatore FROM Progetto WHERE nome =
nomeProgettoI);
    if (progettoEsistente AND autoreEsistente AND testoNonVuoto and nonCreatore) then
        set esito = 1;
        INSERT INTO Commento (data, testo, emailUtente, nomeProgetto) VALUES (current_date(),
testoI, emailAutoreI, nomeProgettoI);
    else
        set esito = 0;
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE InserimentoCandidatura(IN nomeProfiloI VARCHAR(100), IN nomeProgettoI VARCHAR(255),
IN emailUtenteI VARCHAR(255), OUT esito INT)
BEGIN
    declare correttezzaNomeEProgetto boolean;
    declare correttezzaEmail boolean;
    declare candidaturaGiaPresente boolean default false;
    declare candidaturaGiaAccettata boolean default false;
    declare postiDisponibili boolean default true;
    declare candidatoNonCreatore boolean;
    -- variabili da usare in fase di controllo della correttezza della candidatura
    declare livelloRichiesto int;
    declare livelloPosseduto int;
    declare nomeSkillRichiesta varchar(100);
    -- di default l'utente possiede tutte le skill richieste con il livello minimo
    declare skillsCorrette boolean default true;

```

```

declare fineCursor boolean default false;
-- cursore che scorre tutte le skill richieste dal profilo con il loro livello minimo richiesto
declare cursore_skillRichiestaProfilo CURSOR FOR SELECT Sk_r.nomeSkill, Sk_r.livello
        from Skill_Requisito Sk_r join Profilo P
        on Sk_r.nomeProfilo = nomeProfiloI and Sk_r.nomeProgetto =
nomeProgettoI;
-- quando il cursore non trova più righe questa variabile viene impostata a true
declare continue handler for not found set fineCursor = true;
-- il campo profilo deve coincidere un profilo esistente nel sistema
set correttezzaNomeEProgetto = (SELECT count(*) from Profilo where Profilo.nome = nomeProfiloI
and Profilo.nomeProgetto = nomeProgettoI) > 0;
-- il campo email dell'utente deve esistere
set correttezzaEmail = (SELECT count(*) from Utente where Utente.email = emailUtenteI) > 0;
-- non deve esistere già una candidatura identica ancora aperta nel db
set candidaturaGiaPresente = (SELECT count(*) from Candidatura C where
        C.nomeProfilo = nomeProfiloI and
        C.nomeProgetto = nomeProgettoI and
        C.emailUtente = emailUtenteI and
        C.stato = 'aperta') > 0;
-- non deve esistere nel db una candidatura identica già accettata
set candidaturaGiaAccettata = (SELECT count(*) from Candidatura C where
        C.nomeProfilo = nomeProfiloI and
        C.nomeProgetto = nomeProgettoI and
        C.emailUtente = emailUtenteI and
        C.stato = 'chiusa' and C.accettata = 1) > 0;
-- per poter candidarsi a un profilo ci devono essere dei posti ancora disponibili
set postiDisponibili = (SELECT count(*) from Profilo P
        where P.nome = nomeProfiloI and
        P.nomeProgetto = nomeProgettoI and
        P.numero_posizioni > 0) > 0;
set candidatoNonCreatore = emailUtenteI NOT IN (SELECT emailCreatore FROM Progetto WHERE nome =
nomeProgettoI); -- il candidato non deve essere il creatore del progetto a cui si candida
-- se la candidatura è già presente ritorno dei codici di errori che mi serviranno per mostrare
messaggi personalizzati all'utente
if(candidaturaGiaPresente) then
    set esito = 2;
end if;
if(candidaturaGiaAccettata) then
    set esito = 3;
end if;
if(candidatoNonCreatore and correttezzaNomeEProgetto and correttezzaEmail and not
candidaturaGiaPresente and not candidaturaGiaAccettata and postiDisponibili) then
    -- inizia il ciclo che scorre tutte le skill possedute
    open cursore_skillRichiestaProfilo;
    ciclo_skill: loop
        fetch cursore_skillRichiestaProfilo into nomeSkillRichiesta, livelloRichiesto;
        if(fineCursor) then
            leave ciclo_skill;
        end if;
        -- leggo il livello posseduto dell'i-esima skill dell'utente (se non è posseduta assume
0)
        set livelloPosseduto = COALESCE((SELECT Sk_p.livello from Skill_Possesso Sk_p
            where Sk_p.emailUtente = emailUtenteI and
            Sk_p.nomeSkill = nomeSkillRichiesta), 0);
        -- se il livello non è almeno uguale a quello richiesto mi salvo questa info ed esco dal
ciclo
        if(livelloPosseduto < livelloRichiesto) then
            set skillsCorrette = false;
            leave ciclo_skill;
        end if;
    end loop;
    close cursore_skillRichiestaProfilo;
    -- se tutte le skill possedute hanno un livello minimo superiore a quello richiesto dalle
skill del profilo la candidatura
    -- è accettabile e quindi la si inserisce
    if(skillsCorrette) then
        SET esito = 1;
        INSERT INTO Candidatura (stato, nomeProfilo, nomeProgetto, emailUtente)
            values ('aperta', nomeProfiloI, nomeProgettoI, emailUtenteI);
    else
        SET esito = 0;
    end if;
end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE CreazioneProgetto(IN nomeI VARCHAR(255), IN dataLimiteI VARCHAR(255), IN descrI
VARCHAR(255), IN budgetI VARCHAR(255), IN TipoI VARCHAR(255), IN emailCreatoreI VARCHAR(255), OUT
esito INT)
BEGIN
    declare correttezzaNome boolean;
    declare correttezzaBudget boolean;

```

```

declare correttezzaData boolean;
declare correttezzaEmailCreatore boolean;
declare correttezzaTipo boolean;
declare nonEsistente boolean;
set correttezzaData = false;
-- il nome deve essere compilato e non già presente
set correttezzaNome = (LENGTH(nomeI) > 0 AND (nomeI IS NOT NULL));
-- controllo che il campo budget sia convertibile in un numero e che sia positivo
set correttezzaBudget = (budgetI REGEXP '^[0-9]+(\.[0-9]{1,2})?$', AND CAST(budgetI AS
DECIMAL(10,2)) > 0);
-- controllo che il campo data sia in formato YYYY-MM-DD
if(dataLimiteI REGEXP '^[0-9]{2,4}-[0-9]{1,2}-[0-9]{1,2}$') then
-- controllo che sia un valore convertibile in data
if(STR_TO_DATE(dataLimiteI, '%Y-%m-%d') is not null) then
-- controllo che la data inserita sia futura
if(dataLimiteI > curdate()) then
set correttezzaData = true;
end if;
end if;
end if;
-- il creatore del progetto deve esistere
set correttezzaEmailCreatore = (SELECT COUNT(*) FROM Creatore WHERE emailCreatore =
emailCreatoreI) > 0;
-- il tipo deve essere o hardware o software
set correttezzaTipo = (tipoI IN ('Hardware', 'Software'));
-- il progetto non deve già esistere
set nonEsistente = nomeI NOT IN (SELECT nome FROM Progetto);
if (correttezzaNome and correttezzaBudget and correttezzaData and correttezzaEmailCreatore and
correttezzaTipo and nonEsistente) then
set esito = 1;
INSERT IGNORE INTO Progetto VALUES (nomeI, CURDATE(), dataLimiteI, IFNULL(NULLIF(descrI,
'), 'descrizione assente'), 'aperto', budgetI, TipoI, emailCreatoreI);
else
set esito = 0;
end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE CreazioneReward(IN urlFotoI TEXT, IN descrI TEXT, IN nomeI VARCHAR(255), OUT esito
INT)
BEGIN
declare correttezzaFoto boolean;
declare correttezzaNomeProg boolean;
declare fotoNuova boolean;
-- la foto deve avere una estensione valida
set correttezzaFoto = (urlFotoI REGEXP '\\.(jpg|jpeg|png|webp|avif)$');
-- se la query ritorna zero significa che non esiste alcun progetto con quel determinato nome e
quindi non è possibile
-- creare la reward
set correttezzaNomeProg = (SELECT count(*) from Progetto where Progetto.nome = nomeI) > 0;
set fotoNuova = urlFotoI NOT IN (SELECT urlFoto FROM Reward WHERE nomeProgetto = nomeI);

if(correttezzaFoto and correttezzaNomeProg and fotoNuova) then
-- se descrI è vuoto o null imposto di default la descrizione
set esito = 1;
INSERT INTO Reward (urlFoto, descr, nomeProgetto) values (urlFotoI, IFNULL(NULLIF(descrI,
'), 'descrizione assente'), nomeI);
else
set esito = 0;
end if;
END
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE rispondiACommento(IN idCommentoI VARCHAR(255), IN contenutoI TEXT, IN
emailCreatoreI VARCHAR(255), OUT esito INT)
BEGIN
declare correttezzaCommento boolean;
declare correttezzaContenuto boolean;
declare correttezzaEmail boolean;
-- il codice deve essere un numero positivo (utilizzo di espressione regolare che mi ritorna
vero se il codiceI è un numero)
-- (unsigned ammette interi senza segno quindi per forza positivi)
set correttezzaCommento = (idCommentoI REGEXP '^[0-9]+$' and CAST(idCommentoI AS UNSIGNED) > 0);
-- se descrI è una stringa vuota la imposto a null altrimenti restituisce il valore stesso (uso
di nullif)
-- sul risultato interno poi verifico ancora se esso è null -> se è null ritorno false
altrimenti true (uso di ifnull)
set correttezzaContenuto = IF(contenutoI = '' or contenutoI is null, false, true);
-- se la query ritorna zero significa che non esiste nessun utente creatore con quell'email che
ha creato un progetto
set correttezzaEmail = (SELECT count(*) from Progetto where Progetto.emailCreatore =

```

```

emailCreatoreI) > 0;
    if(correttezzaCommento and correttezzaContenuto and correttezzaEmail) then
        set esito = 1;
        INSERT IGNORE INTO Risposta values (idCommentoI, contenutoI, emailCreatoreI);
    else
        set esito = 0;
    end if;
END
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE InserimentoProfilo(IN nomeProfiloI VARCHAR(100), IN nomeProgettoI VARCHAR(255), IN
numeroPosizioniI VARCHAR(255), IN skillsRichiestaI text, OUT esito INT)
BEGIN
    declare correttezzaProfilo boolean;
    declare correttezzaProgetto boolean;
    declare correttezzaNPosizioni boolean;
    declare correttezzaLivello boolean;
    declare correttezzaSkill boolean;
    declare profiloEsistente boolean;
    -- nome dell'i-esima skill della lista passata come parametro alla procedura
    declare skillNome varchar(100);
    declare livelloSkill int;
    declare fineCursor boolean default false;
    -- definisco un cursore che scorrerà una tabella con n righe e una colonna contenente le skills
    -- passate come parametro
    -- il metodo json_table prende in input un oggetto json e ne ritorna una tabella
    declare skillCursor cursor for
        select skill, livello from json_table(skillsRichiestaI,'$[*]'
            columns (skill varchar(100) path '$.nomeSkill', livello varchar(1) path '$.livello')) as
tabella;
    -- quanto il cursore non troverà righe da leggere imposterà la variabile booleana fineCursor a
true
    declare continue handler for not found set fineCursor = true;
    -- il campo del profilo deve essere compilato e non deve contenere esclusivamente cifre
    set correttezzaProfilo = IF(nomeProfiloI = '' or nomeProfiloI is null or nomeProfiloI REGEXP
'^[0-9]+$' , false, true);
    -- se la query ritorna zero significa che non esiste nessun progetto avente il nome specificato
    set correttezzaProgetto = (SELECT count(*) from Progetto where Progetto.nome = nomeProgettoI and
Progetto.tipoProgetto = "Software") > 0;
    -- il campo delle posizioni deve essere un numero intero positivo (unsigned ammette interi senza
segno quindi per forza positivi)
    set correttezzaNPosizioni = (numeroPosizioniI REGEXP '^[0-9]+$' and CAST(numeroPosizioniI AS
UNSIGNED));
    set profiloEsistente = nomeProfiloI IN (SELECT nome FROM Profilo WHERE nomeProgetto =
nomeProgettoI);
    if(correttezzaProfilo and correttezzaProgetto and correttezzaNPosizioni and !profiloEsistente)
then
        INSERT IGNORE INTO Profilo VALUES (nomeProfiloI, nomeProgettoI, numeroPosizioniI);
        OPEN skillCursor;
        skill_loop: loop
            fetch skillCursor into skillNome, livelloSkill;
            -- se non ci sono ulteriori righe da leggere nella tabella esco dal ciclo
            if(fineCursor) then
                leave skill_loop;
            end if;
            -- controllo se la skill esiste
            set correttezzaSkill = (SELECT count(*) from Skill S where S.nome = skillNome) > 0;
            -- il campo livello deve essere un numero intero compreso tra 1 e 5
            set correttezzaLivello = (livelloSkill REGEXP '^[0-9]+$' and CAST(livelloSkill AS
UNSIGNED) and livelloSkill < 6);
            if(correttezzaSkill and correttezzaLivello) then
                set esito = 1;
                INSERT INTO Skill_Requisito VALUES (skillNome, nomeProfiloI, nomeProgettoI,
livelloSkill);
            else
                SET esito = 0;
                leave skill_loop;
            end if;
        end loop;
        CLOSE skillCursor;
    else
        set esito = 0;
    end if;
END
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE checkCandidatura(IN nomeProfiloI VARCHAR(100), IN nomeProgettoI VARCHAR(255), IN
emailUtenteI VARCHAR(255), IN sceltaCreatoreI tinyint(1), OUT esito INT)
BEGIN
    declare correttezzaNomeEProgetto boolean;
    declare correttezzaEmail boolean;
    declare candidaturaGiaPresente boolean default false;

```

```

declare correttezzaSceltaCreatore boolean default false;
declare postiDisponibili boolean default true;
-- il campo profilo deve coincidere un profilo esistente nel sistema
set correttezzaNomeEProgetto = (SELECT count(*) from Profilo where Profilo.nome = nomeProfiloI
and Profilo.nomeProgetto = nomeProgettoI) > 0;
-- il campo email dell'utente deve esistere
set correttezzaEmail = (SELECT count(*) from Utente where Utente.email = emailUtenteI) > 0;
-- deve esistere già una candidatura identica ancora aperta nel db
set candidaturaGiaPresente = (SELECT count(*) from Candidatura C where
                                C.nomeProfilo = nomeProfiloI and
                                C.nomeProgetto = nomeProgettoI and
                                C.emailUtente = emailUtenteI and
                                C.stato = 'aperta') > 0;
-- per poter candidarsi a un profilo ci devono essere dei posti ancora disponibili
set postiDisponibili = (SELECT count(*) from Profilo P
                        where P.nome = nomeProfiloI and
                        P.nomeProgetto = nomeProgettoI and
                        P.numero_posizioni > 0) > 0;
if(sceltaCreatoreI in (0,1)) then
    set correttezzaSceltaCreatore = true;
end if;
if (correttezzaNomeEProgetto and correttezzaEmail and correttezzaSceltaCreatore and
candidaturaGiaPresente) then
    IF (sceltaCreatoreI = 1 and postiDisponibili) or (sceltaCreatoreI <> 1) then
        update Candidatura C
        set C.accettata = sceltaCreatoreI, C.stato = 'chiusa'
        where C.nomeProfilo = nomeProfiloI
            and C.nomeProgetto = nomeProgettoI
            and C.emailUtente = emailUtenteI
            and C.stato = 'aperta';
        set esito = 1;
    else
        set esito = 0;
    end if;
else
    set esito = 0;
end if;
end
$ DELIMITER ;
-- Inserimento di una competenza da parte di un utente amministratore
DELIMITER $
CREATE PROCEDURE InserimentoCompetenza(IN emailAmministratoreI VARCHAR(255), IN nomeCompetenzaI
VARCHAR(255), OUT esito INT)
BEGIN
    declare amministratoreEsistente boolean;
    declare competenzaEsistente boolean;
    -- Controllo se l'utente è un amministratore
    set amministratoreEsistente = (SELECT COUNT(*) FROM Amministratore WHERE emailAmministratore =
emailAmministratoreI) > 0;
    -- Controllo se la competenza non è già presente all'interno del sistema
    set competenzaEsistente = (SELECT COUNT(*) FROM Skill WHERE nome = nomeCompetenzaI) > 0;
    if (NOT(amministratoreEsistente) OR (competenzaEsistente)) then
        -- Restituisco 0 che indica che l'utente non è un amministratore o la competenza è già
esistente
        set esito = 0;
    else
        INSERT INTO Skill (nome) VALUES (nomeCompetenzaI);
        -- Restituisco 1 che indica che l'inserimento della competenza è andata bene
        set esito = 1;
    end if;
END;
$ DELIMITER ;
DELIMITER $
CREATE PROCEDURE InserimentoFotoProgetto(IN urlFotoI VARCHAR(100), IN descrizioneI VARCHAR(100), IN
nomeProgettoI VARCHAR(50), OUT esito INT)
BEGIN
    declare urlCorretto boolean;
    declare progettoEsistente boolean;
    declare fotoNuova boolean;
    -- la foto deve avere una estensione valida
    set urlCorretto = (urlFotoI REGEXP '\\.(jpg|jpeg|png|webp|avif)$');
    -- se la query ritorna zero significa che non esiste alcun progetto con quel determinato nome e
quindi non è possibile
    -- creare la reward
    set progettoEsistente = (SELECT count(*) from Progetto where Progetto.nome = nomeProgettoI) > 0;
    set fotoNuova = urlFotoI NOT IN (SELECT urlImmagine FROM Foto_Progetto WHERE nomeProgetto =
nomeProgettoI);
    if(urlCorretto AND progettoEsistente AND fotoNuova) then
        set esito = 1;
        INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES (descrizioneI,
nomeProgettoI, urlFotoI);
    else

```

```

        set esito = 0;
    end if;
END;
$ DELIMITER ;
-- Inserimento di una componente di tipo Hardware
DELIMITER $
CREATE PROCEDURE InserimentoComponenteHardware(IN nomeComponenteI VARCHAR(255), IN nomeProgettoI
VARCHAR(255), IN descrI VARCHAR(255), IN prezzoI VARCHAR(255), IN quantitaI VARCHAR(255), OUT esito
INT)
BEGIN
    declare progettoEsistenteHardware boolean;
    declare componenteEsistente boolean;
    declare nomeIdoneo boolean;
    -- il nome deve essere compilato e non deve contenere esclusivamente cifre
    set nomeIdoneo = IF(nomeComponenteI = '' or nomeComponenteI is null or nomeComponenteI REGEXP
'^[0-9]+$' , false, true);
    -- Controllo se il progetto è esistente e se è di tipo "Hardware"
    set progettoEsistenteHardware = (SELECT COUNT(*) FROM Progetto WHERE nome = nomeProgettoI AND
tipoProgetto = "Hardware") > 0;
    -- Controllo se il componente è già esistente per il progetto (non possono esserci duplicati ma
per lo stesso componente possono esserci più quantità)
    set componenteEsistente = (SELECT COUNT(*) FROM Componente WHERE nome = nomeComponenteI AND
nomeProgetto = nomeProgettoI) > 0;
    if ((NOT(progettoEsistenteHardware)) OR (componenteEsistente) OR (NOT(nomeIdoneo))) then
        -- Restituisco 0 che indica che il progetto non esiste o il progetto è software o il
componente hardware è già presente
        set esito = 0;
    else
        -- Cast dei parametri
        SET prezzoI = CAST(prezzoI AS DECIMAL(10,2));
        SET quantitaI = CAST(quantitaI AS UNSIGNED);
        INSERT INTO Componente (nome, nomeProgetto, prezzo, descr, quantita) VALUES
(nomeComponenteI, nomeProgettoI, prezzoI, descrI, quantitaI);
        -- Restituisco 1 che indica che l'inserimento della componente hardware è andata a buon fine
        set esito = 1;
    end if;
END;
$ DELIMITER ;
-- Statistiche che vengono calcolate tramite le viste
-- Vista che visualizza la classifica degli utenti creatori, in base al loro valore di affidabilità
(mostra solo i primi 3 nickname)
CREATE VIEW ClassificaCreatoriAffidabilita AS
SELECT Utente.nickname, Creatore.affidabilita
FROM Utente
JOIN Creatore ON Utente.email = Creatore.emailCreatore
-- Ordino i creatori in base all'affidabilità in ordine decrescente
ORDER BY Creatore.affidabilita DESC
-- Restituisco solo i primi 3 come richiesto dalla consegna
LIMIT 3;
-- Vista che visualizza i progetti aperti che sono più vicini al completamento, minore differenza
tra budget avvio e somma totale dei finanziamenti ricevuti (mostra solo i primi 3 progetti)
CREATE VIEW ProgettiApertiCompletamentoFinanziamento AS
-- Calcolo la differenza tra budget e fondi ricevuti. COALESCE indica che se il progetto non ha
ricevuto finanziamenti, la funzione non restituisce null ma 0
SELECT Progetto.nome, (Progetto.budget_avvio - COALESCE(SUM(Finanziamento.importo), 0)) AS
budget_mancante, Progetto.budget_avvio
FROM Progetto
-- Utilizzo il LEFT JOIN per visualizzare anche eventuali progetti che non hanno ancora ricevuto un
finanziamento (valore 0)
LEFT JOIN Finanziamento ON Progetto.nome = Finanziamento.nomeProgetto
WHERE Progetto.stato = "aperto"
GROUP BY Progetto.nome, Progetto.budget_avvio
-- Ordino in ordine crescente per vedere il budget dei progetti più vicini al completamento
ORDER BY budget_mancante ASC
LIMIT 3;
-- Vista che visualizza la classifica degli utenti ordinati in base al totale di finanziamenti
erogati (mostra solo i primi 3 nickname)
CREATE VIEW ClassificaUtentiFinanziatori AS
-- Sommo il totale dei vari finanziamenti del singolo utente con COALESCE che, nel caso non siano
presenti finanziamenti, non restituisce null ma 0
SELECT Utente.nickname, COALESCE(SUM(Finanziamento.importo), 0) AS totale_finanziamento
FROM Utente
-- Utilizzo il LEFT JOIN per visualizzare anche eventuali utenti che non hanno ancora fatto un
finanziamento (valore 0)
LEFT JOIN Finanziamento ON Utente.email = Finanziamento.emailUtente
GROUP BY Utente.nickname
-- Ordino in modo decrescente per trovare quali utenti hanno finanziato di più
ORDER BY totale_finanziamento DESC
LIMIT 3;
-- Popolamento degli utenti che possono accedere al sistema: utente normale, utente creatore e
utente amministratore
CALL RegistrazioneAmministratore('mario.rossi@email.com', 'pass1234', 'Mario', 'Rossi', 'Roma',

```

```

'1980', 'mario1980', '1001', @esito);
CALL RegistrazioneNormale('normal.user@email.com', 'pass1234', 'User', 'Normal', 'Bologna', '2000',
'normalUser', @esito);
CALL RegistrazioneCreatore('giulia.bianchi@email.com', 'securePass', 'Giulia', 'Bianchi', 'Milano',
'1990', 'giulietta90', @esito);
CALL RegistrazioneCreatore('federica.verdi@email.com', 'securePass', 'Federica', 'Verdi', 'Napoli',
'2000', 'fede00', @esito);
CALL RegistrazioneCreatore('lucia.gialli@email.com', 'securePass', 'Lucia', 'Gialli', 'Torino',
'1998', 'luci98', @esito);
-- Popolamento dei progetti di tipo Hardware e Software -- DA FINIRE
CALL CreazioneProgetto('DriveSenseAI', '2025-10-10', 'Progetto software innovativo per auto con AI',
'500000.00', 'Software', 'giulia.bianchi@email.com', @esito);
CALL CreazioneProgetto('CompanyTagline', '2026-01-01', 'Progetto software per creare delle
fotografie avanzate', '100000.00', 'Software', 'giulia.bianchi@email.com', @esito);
CALL CreazioneProgetto('ProgettoNatura', '2027-02-02', 'Progetto per la salvaguardia della natura',
'50000.00', 'Hardware', 'giulia.bianchi@email.com', @esito);
CALL CreazioneProgetto('PulseTech', '2025-08-08', 'Progetto per creare uno smartwatch avanzato con
AI per salute e fitness', '300000.00', 'Hardware', 'federica.verdi@email.com', @esito);
CALL CreazioneProgetto('EcoSmartHome', '2025-07-07', 'Progetto per costruire una casa smart
ecosostenibile', '1000000.00', 'Hardware', 'lucia.gialli@email.com', @esito);
CALL CreazioneProgetto('VisaAI', '2026-03-03', 'Progetto per migliorare la sicurezza dei pagamenti
con AI', '200000.00', 'Software', 'lucia.gialli@email.com', @esito);
-- Popolamento delle immagini dei progetti creati
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('primo logo progetto
DriveSenseAI', 'DriveSenseAI', 'public/immagini/progetti/DriveSenseAI/fotoProgetto/progetto1.webp');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('secondo logo progetto
DriveSenseAI', 'DriveSenseAI', 'public/immagini/progetti/DriveSenseAI/fotoProgetto/progetto2.png');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('immagine telecamera per
drive sense', 'DriveSenseAI', 'public/immagini/progetti/DriveSenseAI/fotoProgetto/progetto3.jpeg');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('logo progetto
CompanyTagline', 'CompanyTagline',
'public/immagini/progetti/CompanyTagline/fotoProgetto/progetto4.avif');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('immagine fotocamera con
AI', 'CompanyTagline', 'public/immagini/progetti/CompanyTagline/fotoProgetto/progetto5.jpg');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('logo progetto
ProgettoNatura', 'ProgettoNatura',
'public/immagini/progetti/ProgettoNatura/fotoProgetto/progetto6.png');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('immagine natura da
salvaguardare', 'ProgettoNatura',
'public/immagini/progetti/ProgettoNatura/fotoProgetto/progetto7.jpg');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('logo progetto PulseTech',
'PulseTech', 'public/immagini/progetti/PulseTech/fotoProgetto/progetto8.webp');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('immagine di un
dispositivo pulsetech', 'PulseTech',
'public/immagini/progetti/PulseTech/fotoProgetto/progetto9.webp');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('logo progetto
EcoSmartHome', 'EcoSmartHome', 'public/immagini/progetti/EcoSmartHome/fotoProgetto/progetto10.png');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('immagine funzionamento
della nuova EcoSmartHome', 'EcoSmartHome',
'public/immagini/progetti/EcoSmartHome/fotoProgetto/progetto11.jpeg');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('immagine di una casa di
tipo EcoSmartHome', 'EcoSmartHome',
'public/immagini/progetti/EcoSmartHome/fotoProgetto/progetto12.jpeg');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('primo logo progetto
VisaAI', 'VisaAI', 'public/immagini/progetti/VisaAI/fotoProgetto/progetto13.webp');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('secondo logo progetto
VisaAI', 'VisaAI', 'public/immagini/progetti/VisaAI/fotoProgetto/progetto14.jpeg');
INSERT INTO Foto_Progetto(descrizione, nomeProgetto, urlImmagine) VALUES ('funzionalità contactless
carta Visa con AI', 'VisaAI', 'public/immagini/progetti/VisaAI/fotoProgetto/progetto15.jpeg');
-- Popolamento componenti Hardware dei progetti di tipo Hardware
CALL InserimentoComponenteHardware('Drone di Monitoraggio Fauna', 'ProgettoNatura', 'Drone con
telecamere AI per il monitoraggio degli animali', '5000.00', '3', @esito);
CALL InserimentoComponenteHardware('Collare GPS Intelligente', 'ProgettoNatura', 'Dispositivo per
tracciare il movimento degli animali selvatici', '500.00', '20', @esito);
CALL InserimentoComponenteHardware('Sensore Acustico per Bracconaggio', 'ProgettoNatura', 'Microfono
AI per rilevare spari e movimenti sospetti', '1000.00', '5', @esito);
CALL InserimentoComponenteHardware('Telecamera Notturna per Monitoraggio', 'ProgettoNatura',
'Telecamera a infrarossi per registrare attività notturna', '1500.00', '6', @esito);
CALL InserimentoComponenteHardware('Sensore di Integrità dei Cavi', 'PulseTech', 'Dispositivo per il
monitoraggio della funzionalità dei cavi', '100.00', '100', @esito);
CALL InserimentoComponenteHardware('Batteria a Lunga Durata', 'PulseTech', 'Batteria ottimizzata per
garantire una durata di oltre 7 giorni', '1000.00', '80', @esito);
CALL InserimentoComponenteHardware('Pannelli Solari Avanzati', 'EcoSmartHome', 'Pannelli
fotovoltaici ad alta efficienza per autonomia energetica', '10000.00', '5', @esito);
CALL InserimentoComponenteHardware('Sistema Domotico AI', 'EcoSmartHome', 'Sistema AI per la
gestione intelligente degli elettrodomestici', '15000.00', '4', @esito);
CALL InserimentoComponenteHardware('Sistema di Recupero Acque Piovane', 'EcoSmartHome', 'Impianto
per raccogliere e riutilizzare acqua piovana', '5000.00', '10', @esito);
-- Popolamento delle reward dei progetti creati
CALL CreazioneReward('public/immagini/progetti/DriveSenseAI/fotoRewards/reward1.webp', 'buono sconto
amazon di 15€', 'DriveSenseAI', @esito);
CALL CreazioneReward('public/immagini/progetti/DriveSenseAI/fotoRewards/reward2.avif', 'buono sconto

```

```

sephora da 20€', 'DriveSenseAI', @esito);
CALL CreazioneReward('public/immagini/progetti/DriveSenseAI/fotoRewards/reward3.png', 'buono sconto
unieuro da 10€', 'DriveSenseAI', @esito);
CALL CreazioneReward('public/immagini/progetti/DriveSenseAI/fotoRewards/reward4.jpg', 'buono sconto
conad di 5€', 'DriveSenseAI', @esito);
CALL CreazioneReward('public/immagini/progetti/DriveSenseAI/fotoRewards/reward5.jpg', 'buono sconto
adidas di 25€', 'DriveSenseAI', @esito);
CALL CreazioneReward('public/immagini/progetti/CompanyTagline/fotoRewards/reward6.jpg', 'smartbox
gratuita "Adrenalina Pace"', 'CompanyTagline', @esito);
CALL CreazioneReward('public/immagini/progetti/CompanyTagline/fotoRewards/reward7.avif', 'un
biglietto aereo gratuito', 'CompanyTagline', @esito);
CALL CreazioneReward('public/immagini/progetti/ProgettoNatura/fotoRewards/reward8.jpg',
'personalizzazione gratuita di un prodotto', 'ProgettoNatura', @esito);
CALL CreazioneReward('public/immagini/progetti/ProgettoNatura/fotoRewards/reward9.jpg', 'penna del
progetto ProgettoNatura', 'ProgettoNatura', @esito);
CALL CreazioneReward('public/immagini/progetti/ProgettoNatura/fotoRewards/reward10.jpg',
'portachiavi del progetto ProgettoNatura', 'ProgettoNatura', @esito);
CALL CreazioneReward('public/immagini/progetti/PulseTech/fotoRewards/reward10.jpg', 'portachiavi del
progetto PulseTech', 'PulseTech', @esito);
CALL CreazioneReward('public/immagini/progetti/EcoSmartHome/fotoRewards/reward1.webp', 'buono sconto
amazon di 15€', 'EcoSmartHome', @esito);
CALL CreazioneReward('public/immagini/progetti/EcoSmartHome/fotoRewards/reward2.avif', 'buono sconto
sephora da 20€', 'EcoSmartHome', @esito);
CALL CreazioneReward('public/immagini/progetti/EcoSmartHome/fotoRewards/reward5.jpg', 'buono sconto
adidas di 25€', 'EcoSmartHome', @esito);
CALL CreazioneReward('public/immagini/progetti/EcoSmartHome/fotoRewards/reward6.jpg', 'smartbox
gratuita "Adrenalina Pace"', 'EcoSmartHome', @esito);
CALL CreazioneReward('public/immagini/progetti/VisaAI/fotoRewards/reward3.png', 'buono sconto
unieuro da 10€', 'VisaAI', @esito);
CALL CreazioneReward('public/immagini/progetti/VisaAI/fotoRewards/reward5.jpg', 'buono sconto adidas
di 25€', 'VisaAI', @esito);
CALL CreazioneReward('public/immagini/progetti/VisaAI/fotoRewards/reward7.avif', 'un biglietto aereo
gratuito', 'VisaAI', @esito);
-- Popolamento dei finanziamenti del progetto, uno finanziato al 100%, due circa al 70%, due circa
al 30% e uno non ancora finanziato
CALL InserimentoFinanziamento('DriveSenseAI', '200000.00', 'mario.rossi@email.com', @esito);
CALL SceltaReward('1', 'mario.rossi@email.com', 'DriveSenseAI', @esito);
CALL InserimentoFinanziamento('DriveSenseAI', '200000.00', 'federica.verdi@email.com', @esito);
CALL SceltaReward('4', 'federica.verdi@email.com', 'DriveSenseAI', @esito);
CALL InserimentoFinanziamento('ProgettoNatura', '10000.00', 'mario.rossi@email.com', @esito);
CALL SceltaReward('9', 'mario.rossi@email.com', 'ProgettoNatura', @esito);
CALL InserimentoFinanziamento('PulseTech', '100000.00', 'lucia.gialli@email.com', @esito);
CALL SceltaReward('11', 'lucia.gialli@email.com', 'PulseTech', @esito);
CALL InserimentoFinanziamento('PulseTech', '100000.00', 'mario.rossi@email.com', @esito);
CALL SceltaReward('11', 'mario.rossi@email.com', 'PulseTech', @esito);
CALL InserimentoFinanziamento('PulseTech', '100000.00', 'federica.verdi@email.com', @esito);
CALL SceltaReward('11', 'federica.verdi@email.com', 'PulseTech', @esito);
CALL InserimentoFinanziamento('EcoSmartHome', '400000.00', 'giulia.bianchi@email.com', @esito);
CALL SceltaReward('13', 'giulia.bianchi@email.com', 'EcoSmartHome', @esito);
CALL InserimentoFinanziamento('EcoSmartHome', '100000.00', 'normal.user@email.com', @esito);
CALL SceltaReward('14', 'normal.user@email.com', 'EcoSmartHome', @esito);
CALL InserimentoFinanziamento('VisaAI', '50000.00', 'federica.verdi@email.com', @esito);
CALL SceltaReward('16', 'federica.verdi@email.com', 'VisaAI', @esito);
CALL InserimentoFinanziamento('VisaAI', '50000.00', 'lucia.gialli@email.com', @esito);
CALL SceltaReward('17', 'lucia.gialli@email.com', 'VisaAI', @esito);
CALL InserimentoFinanziamento('VisaAI', '50000.00', 'mario.rossi@email.com', @esito);
CALL SceltaReward('18', 'mario.rossi@email.com', 'VisaAI', @esito);
-- Popolamento delle competenze inserite dall'utente amministratore
CALL InserimentoCompetenza('mario.rossi@email.com', 'Conoscenza Lingua Inglese', @esito);
CALL InserimentoCompetenza('mario.rossi@email.com', 'Programmazione in Python', @esito);
CALL InserimentoCompetenza('mario.rossi@email.com', 'Machine Learning', @esito);
CALL InserimentoCompetenza('mario.rossi@email.com', 'Lavorare in Team', @esito);
CALL InserimentoCompetenza('mario.rossi@email.com', 'CyberSecurity', @esito);
-- Popolamento per l'inserimento delle abilità delle competenze degli utenti
CALL InserimentoSkillCurriculum('mario.rossi@email.com', 'Machine Learning', '4', @esito);
CALL InserimentoSkillCurriculum('mario.rossi@email.com', 'Programmazione in Python', '5', @esito);
CALL InserimentoSkillCurriculum('mario.rossi@email.com', 'Lavorare in Team', '2', @esito);
CALL RimozioneSkillCurriculum('mario.rossi@email.com', 'Lavorare in Team', @esito);
CALL InserimentoSkillCurriculum('normal.user@email.com', 'Programmazione in Python', '1', @esito);
CALL InserimentoSkillCurriculum('normal.user@email.com', 'Machine Learning', '4', @esito);
CALL InserimentoSkillCurriculum('normal.user@email.com', 'Lavorare in Team', '5', @esito);
CALL InserimentoSkillCurriculum('giulia.bianchi@email.com', 'Conoscenza Lingua Inglese', '5',
@esito);
CALL InserimentoSkillCurriculum('giulia.bianchi@email.com', 'Programmazione in Python', '3',
@esito);
CALL InserimentoSkillCurriculum('giulia.bianchi@email.com', 'Machine Learning', '5', @esito);
CALL InserimentoSkillCurriculum('giulia.bianchi@email.com', 'Lavorare in Team', '4', @esito);
CALL InserimentoSkillCurriculum('federica.verdi@email.com', 'Conoscenza Lingua Inglese', '3',
@esito);
CALL InserimentoSkillCurriculum('federica.verdi@email.com', 'Machine Learning', '4', @esito);
CALL InserimentoSkillCurriculum('federica.verdi@email.com', 'Lavorare in Team', '4', @esito);
CALL InserimentoSkillCurriculum('lucia.gialli@email.com', 'Programmazione in Python', '4', @esito);

```

```

CALL InserimentoSkillCurriculum('lucia.gialli@email.com', 'Machine Learning', '4', @esito);
CALL InserimentoSkillCurriculum('lucia.gialli@email.com', 'CyberSecurity', '2', @esito);
-- Popolamento per l'inserimento dei profili
CALL InserimentoProfilo('Data Scientist', 'DriveSenseAI', '2', '{"nomeSkill": "Programmazione in Python", "livello": "2"}, {"nomeSkill": "Machine Learning", "livello": "3"}', @esito);
CALL InserimentoProfilo('Business Analyst', 'DriveSenseAI', '1', '{"nomeSkill": "Lavorare in Team", "livello": "3"}', @esito);
CALL InserimentoProfilo('Software Engineer', 'CompanyTagline', '3', '{"nomeSkill": "Conoscenza Lingua Inglese", "livello": "2"}, {"nomeSkill": "Machine Learning", "livello": "3"}', @esito);
CALL InserimentoProfilo('Software Engineer', 'VisaAI', '1', '{"nomeSkill": "Conoscenza Lingua Inglese", "livello": "2"}, {"nomeSkill": "Machine Learning", "livello": "2"}, {"nomeSkill": "Programmazione in Python", "livello": "2"}', @esito);
-- Popolamento per l'inserimento delle candidature alle posizioni disponibili
CALL InserimentoCandidatura('Data Scientist', 'DriveSenseAI', 'lucia.gialli@email.com', @esito);
CALL InserimentoCandidatura('Data Scientist', 'DriveSenseAI', 'mario.rossi@email.com', @esito);
CALL InserimentoCandidatura('Business Analyst', 'DriveSenseAI', 'normal.user@email.com', @esito);
-- La candidatura fallirà in quanto è lei l'utente creatore del progetto
CALL InserimentoCandidatura('Business Analyst', 'DriveSenseAI', 'giulia.bianchi@email.com', @esito);
CALL InserimentoCandidatura('Business Analyst', 'DriveSenseAI', 'federica.verdi@email.com', @esito);
CALL InserimentoCandidatura('Software Engineer', 'CompanyTagline', 'federica.verdi@email.com', @esito);
CALL InserimentoCandidatura('Software Engineer', 'CompanyTagline', 'giulia.bianchi@email.com', @esito);
-- Popolamento per valutare positivamente o negativamente una candidatura arrivate
CALL checkCandidatura('Data Scientist', 'DriveSenseAI', 'mario.rossi@email.com', true, @esito);
CALL checkCandidatura('Data Scientist', 'DriveSenseAI', 'lucia.gialli@email.com', false, @esito);
CALL checkCandidatura('Business Analyst', 'DriveSenseAI', 'federica.verdi@email.com', true, @esito);
-- Popolamento dei commenti al progetto
CALL CommentaProgetto('DriveSenseAI', 'federica.verdi@email.com', 'Intelligenza artificiale sta cambiando il settore automobilistico e DriveSenseAI promette di portare la sicurezza stradale a un nuovo livello. Idea di un sistema avanzato di assistenza alla guida basato su machine learning è davvero interessante. Non vedo il momento di vedere come verrà implementato il rilevamento predittivo degli ostacoli!', @esito);
CALL CommentaProgetto('DriveSenseAI', 'mario.rossi@email.com', 'Sembra un progetto interessante!', @esito);
CALL CommentaProgetto('DriveSenseAI', 'lucia.gialli@email.com', 'Sono un sacco interessato a questa macchina, non vedo ora che sia il momento che sia pronta per poterla acquistare!', @esito);
CALL CommentaProgetto('CompanyTagline', 'federica.verdi@email.com', 'Creare fotografie avanzate con aiuto intelligenza artificiale è una sfida affascinante. Mi piacerebbe sapere di più sulla tecnologia usata per migliorare la qualità dell'immagine e sulle possibili applicazioni nel settore pubblicitario o della fotografia artistica.', @esito);
CALL CommentaProgetto('ProgettoNatura', 'normal.user@email.com', 'Proteggere l'ambiente con la tecnologia è una missione nobile. I droni e i sensori per monitorare la fauna selvatica potrebbero davvero fare la differenza nella lotta contro il bracconaggio. Sarebbe interessante sapere se sono previsti accordi con enti ambientali o università.', @esito);
CALL CommentaProgetto('EcoSmartHome', 'giulia.bianchi@email.com', 'Una casa ecosostenibile e intelligente è un obiettivo essenziale per il futuro. Se i sistemi di automazione possono ridurre i consumi energetici senza compromettere il comfort, questo progetto potrebbe essere un punto di svolta nel settore delle smart home.', @esito);
CALL CommentaProgetto('EcoSmartHome', 'normal.user@email.com', 'Sono un sacco interessato a questa casa, non vedo ora che sia il momento che sia pronta per poterla acquistare!', @esito);
CALL CommentaProgetto('EcoSmartHome', 'mario.rossi@email.com', 'Spettacolare!', @esito);
-- Popolamento della risposta ai commenti del progetto da parte dell'utente creatore del progetto
CALL rispondiACommento(1, 'Grazie Mario! Il nostro focus principale è proprio la sicurezza stradale. Il sistema sarà in grado di analizzare il comportamento degli altri veicoli e dei pedoni in tempo reale, per anticipare possibili pericoli e assistere il guidatore nelle decisioni critiche.', 'giulia.bianchi@email.com', @esito);
CALL rispondiACommento(2, 'Grazie per il supporto!', 'giulia.bianchi@email.com', @esito);
CALL rispondiACommento(5, 'Hai ragione Federica! Stiamo collaborando con diverse ONG e università per testare i dispositivi sul campo. Obiettivo di fornire ai ricercatori strumenti avanzati per il monitoraggio della fauna e la prevenzione delle attività illegali.', 'giulia.bianchi@email.com', @esito);
CALL rispondiACommento(6, 'Grazie della precisazione!', 'lucia.gialli@email.com', @esito);
CALL rispondiACommento(8, 'Verissimo!', 'lucia.gialli@email.com', @esito);

```

ISTRUZIONI D'USO

Requisiti

- Docker installato e correttamente configurato.

- **Visual Studio Code (VSCode)** come editor di testo e per l'esecuzione dei comandi dal terminale.
- **MySQL Workbench** per la gestione del database.

Installazione e Avvio del Progetto

1. Estrazione dell'Archivio

Per iniziare, decomprimere il file del progetto in una cartella locale.

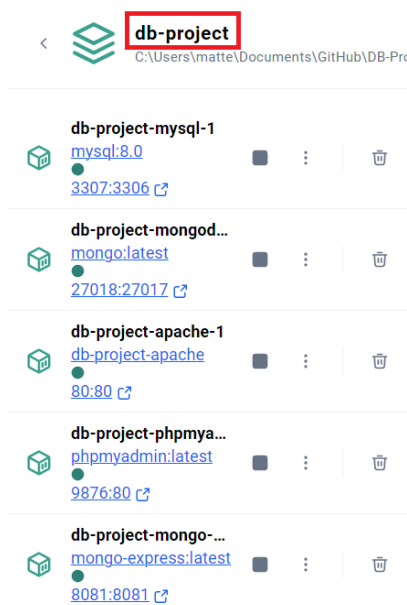
2. Configurazione docker-compose.yaml e config.php

Configurare il file docker-compose.yaml con i dati relativi a numeri di porta, username e password vari. Successivamente, usare i dati inseriti per configurare il file config.php per le connessioni ai database.

3. Avvio del Container Docker

Il progetto utilizza:

- il file docker-compose.yaml per includere tutti gli elementi necessari all'esecuzione del progetto (MySQL, PhpMyAdmin, MongoDB, MongoExpress, Apache)
- il file Dockerfile per l'installazione di tutte le dipendenze necessarie all'esecuzione dell'applicazione.



4. Avvio dell'Applicazione con VSCode

1. Aprire **VSCode** nella cartella del progetto.
2. Avviare il terminale ed eseguire il seguente comando:

docker compose up --build -d

```
PS C:\Users\matte\Documents\GitHub\DB-Project> docker compose up --build -d
Compose now can delegate build to bake for better performances
Just set COMPOSE_BAKE=true
[+] Building 1.2s (10/10) FINISHED
[+] Running 6/6
✓ apache Built
✓ Container db-project-mysql-1 Running
✓ Container db-project-mongodb-1 Running
✓ Container db-project-mongo-express-1 Running
✓ Container db-project-apache-1 Running
✓ Container db-project-phpmyadmin-1 Running
```

Questo comando avvierà l'applicazione per la prima volta.

5. Configurazione del Database

1. Aprire **MySQL Workbench**.
2. Connettersi al database.
3. Eseguire il file **SQL** fornito.

Credenziali Utenti

Il sistema include alcuni utenti predefiniti.

Utente Amministratore

- **Mario Rossi**
 - **Email:** mario.rossi@email.com
 - **Password:** pass1234
 - **Codice di sicurezza:** 1001

Utente Normale

- **User Normal**
 - **Email:** normal.user@email.com
 - **Password:** pass1234

Utenti Creatori

- **Giulia Bianchi**
 - **Email:** giulia.bianchi@email.com
 - **Password:** securePass
 - **Federica Verdi**
 - **Email:** federica.verdi@email.com
 - **Password:** securePass
 - **Lucia Gialli**
 - **Email:** lucia.gialli@email.com
 - **Password:** securePass
-